

Programação Para Dispositivos Móveis II

INTRODUÇÃO À PROGRAMAÇÃO EM SWIFT

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jmr@estg.ipp.pt, Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt

Requisitos

Hardware: Mac com macOS 10.14 ou superior;

Software: Xcode (download gratuito da Mac App Store);

Developer: registo gratuito como developer em <https://developer.apple.com/ios/>



Introdução

Como outras linguagens de programação, o programa apenas executa as instruções do *developer*.

“Often, what you think you’ve told your program to do and what it actually does are two different things.”

(O que pensas que disseste ao teu programa para fazer e aquilo que ele realmente faz, são duas coisas diferentes.)

:Bennett, G., Fisher, M., & Lees, B. (2011). Objective-C for Absolute Beginners: iPhone, iPad and Mac Programming Made Easy.

Desenvolvimento com sucesso

Design requirements

- Ajudam o programador a definir os comportamentos da aplicação, ou seja, identificar o que a App deve ou não fazer;
- Devem ser identificados em colaboração com utilizadores e clientes.

Interface de Utilizador

- A Apple recomenda que mais de 50% do processo de desenvolvimento deve ser dedicado à interface.

Desenvolvimento com sucesso (2)

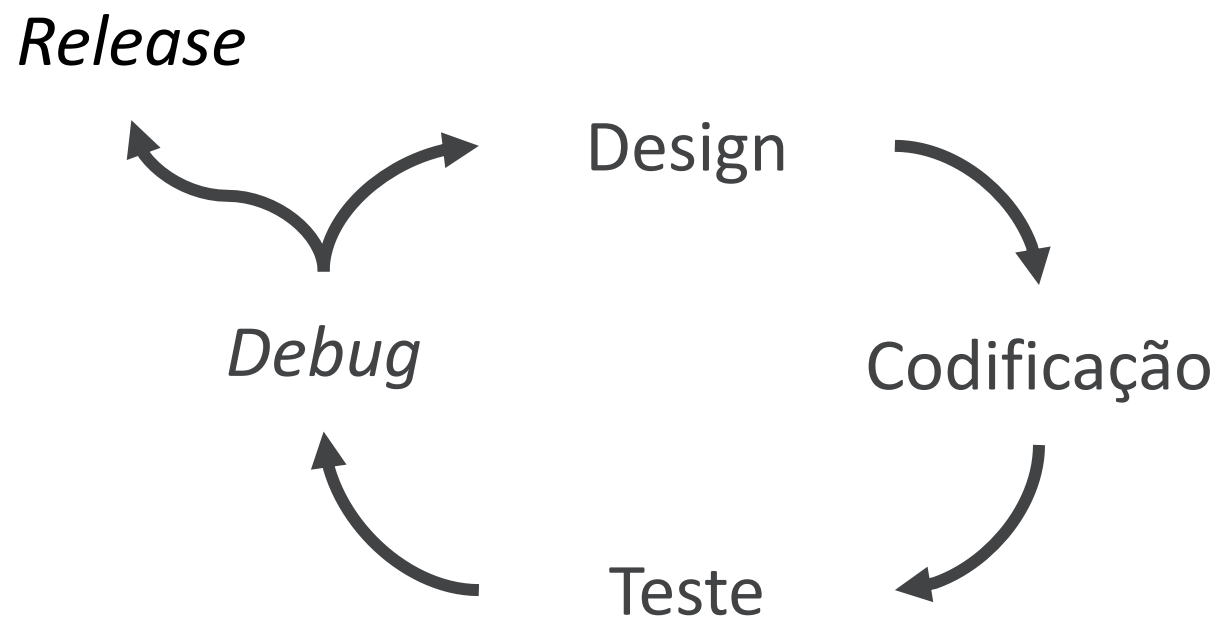
Codificar

- Escrever o código da aplicação de acordo com os requisitos anteriormente identificados.

Testes e debugging

- Identificação de resultados/comportamentos indesejados;
- Deve ser realizado por uma pessoa que não tenha desenvolvido o código.

Desenvolvimento com sucesso (3)



Programação Orientada a Objetos (POO)

- Reutilização de código;
- Manutenção de código;
- Redução do custo de desenvolvimento.

A POO pode ser vista como uma coleção de objetos num programa



Xcode IDE

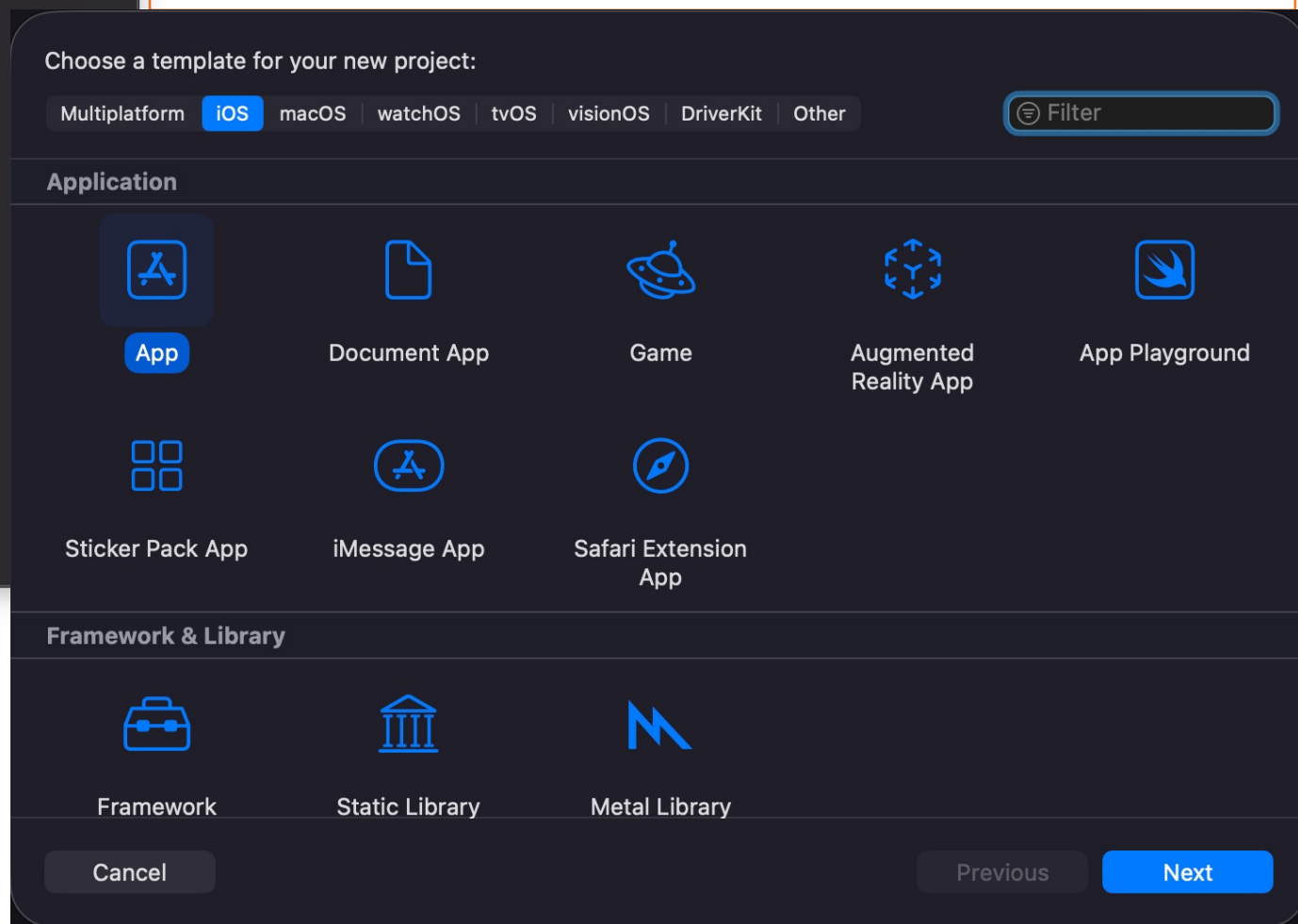
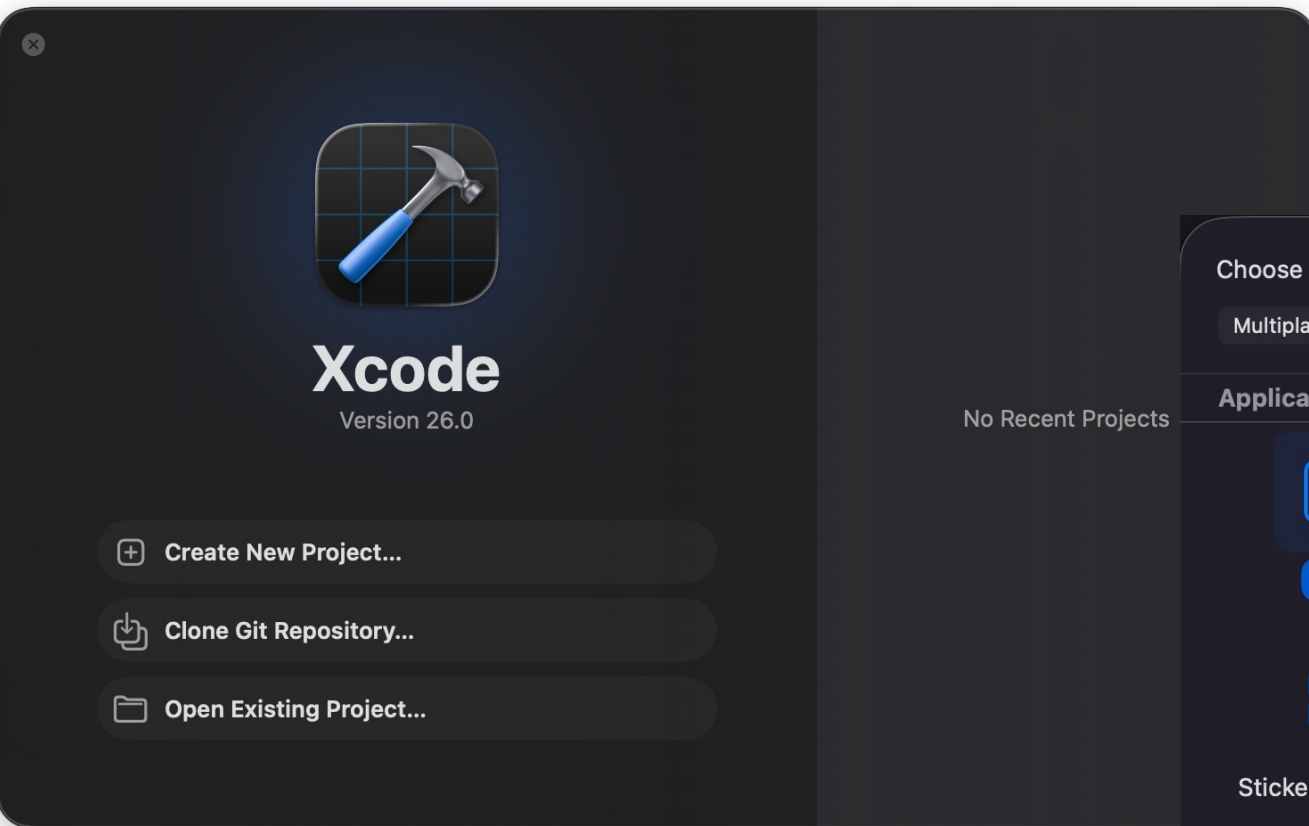
IDE (Integrated Development Environment)

- Desenvolvimento de código;
- *User interface layout*;
- Ferramentas para *debug*;
- Documentação;
- Preview
- Simulador.



Xcode IDE | **template** para novo projeto

- Os **templates** incluem configuração e ficheiros necessários para agilizar o início do desenvolvimento



Xcode IDE | opções do novo projeto

Choose options for your new project:

Product Name: HelloWorld 1

Team: Carlos Aldeias (Personal Team - cfpa@... 2

Organization Identifier: pt.ipp.estg.dwdm.pdm2 3

Bundle Identifier: pt.ipp.estg.dwdm.pdm2.HelloWorld

Interface: Storyboard 4

Language: Swift 5

Storage: None 6

☐ Host in CloudKit

☒ Include Tests 7

Cancel Previous Next 8

1. Nome do produto

- nome da app que ficará visível na AppStore e nos dispositivos após instalação

2. Equipa ou *developer*

- Registado no Apple Developer Program, em <https://developer.apple.com/programs/enroll/>

3. ID da entidade

- reverse DNS string that uniquely identifies the organization

4. Tipo de interface

- Storyboard
- SwiftUI

5. Linguagem de programação

- Swift
- Objective-C

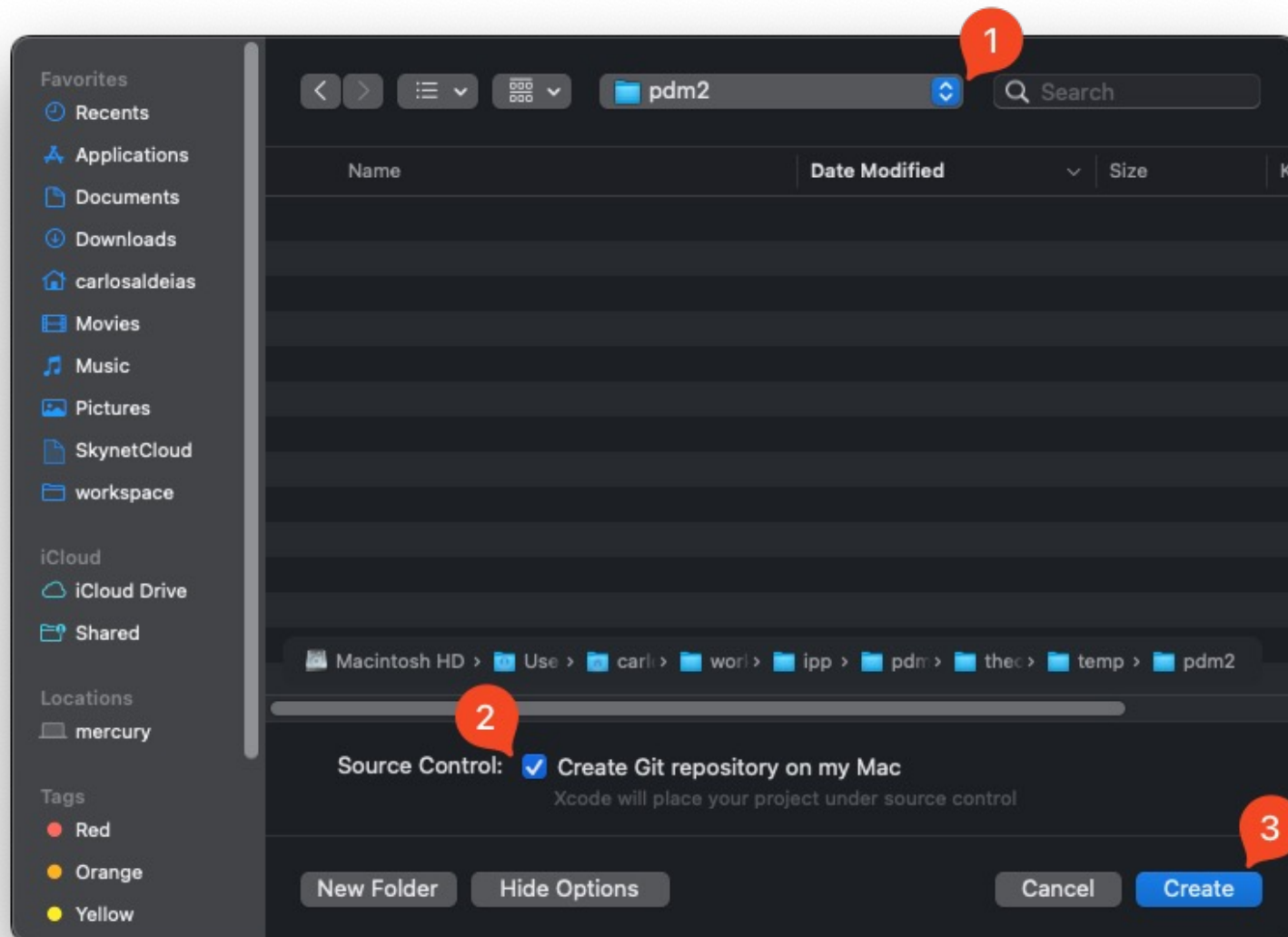
6. Armazenamento

- Suporte para persistência de dados com CoreData

7. Testes

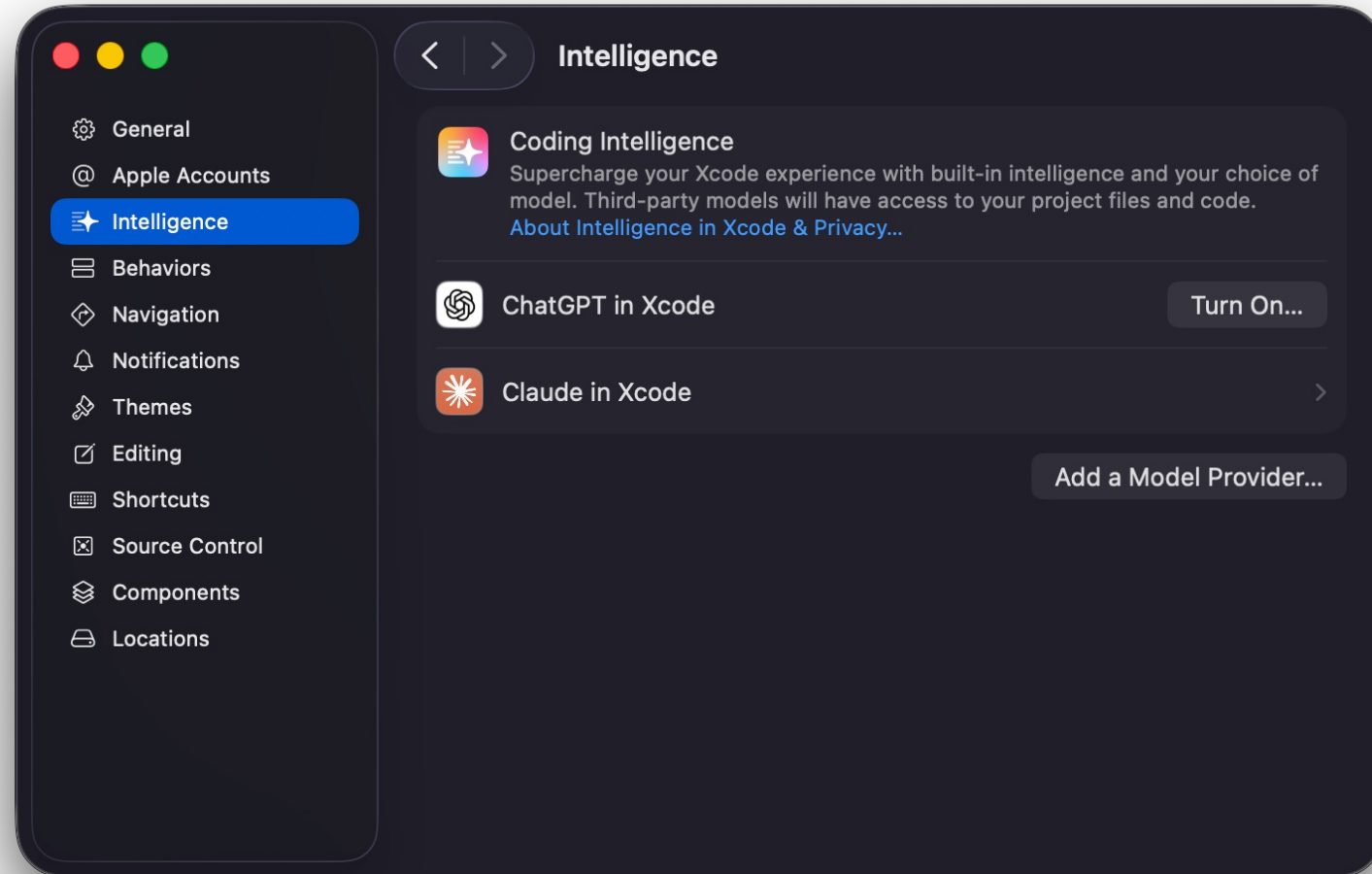
- Suporte para a testes unitários

Xcode IDE | **localização** do projeto



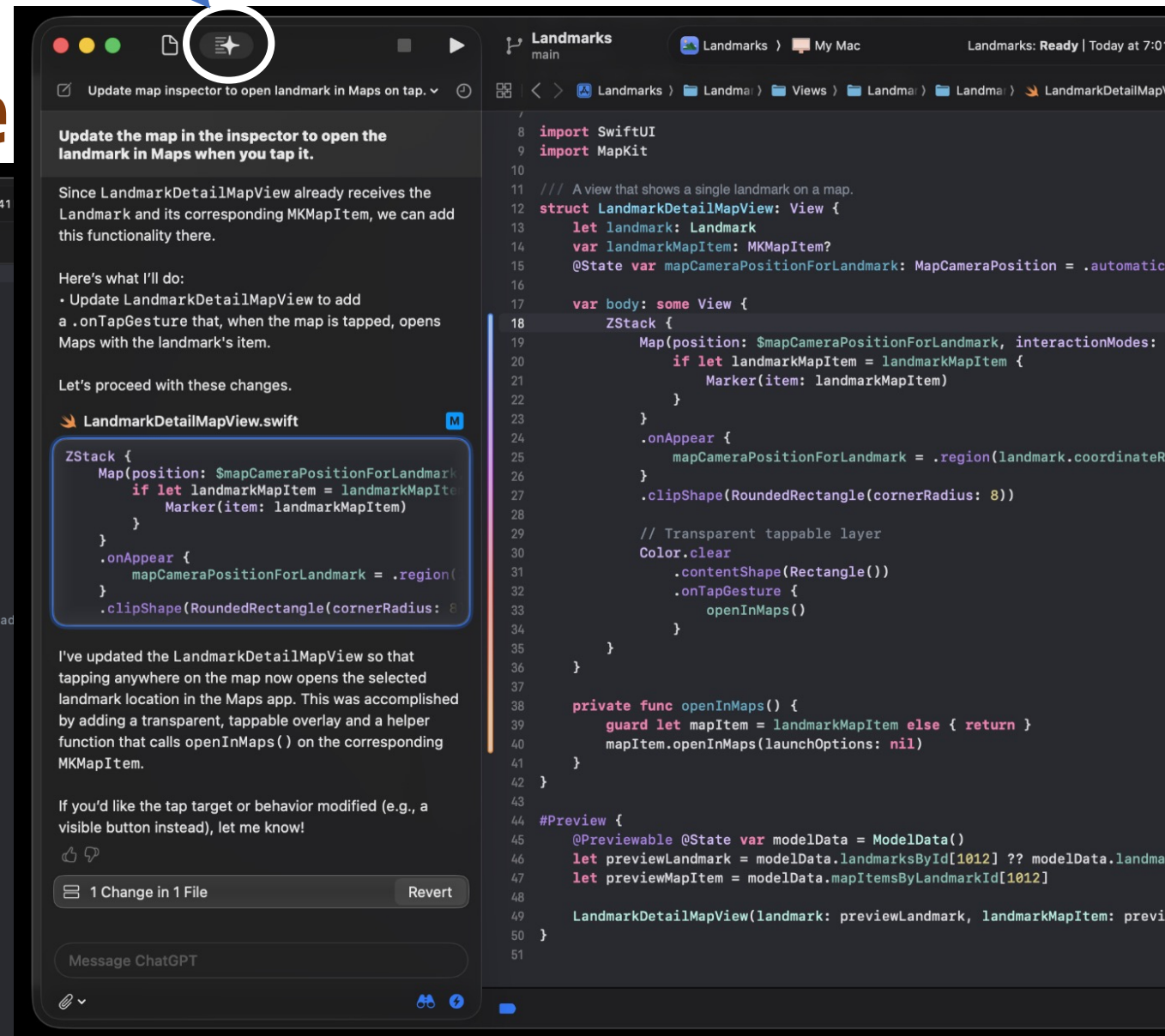
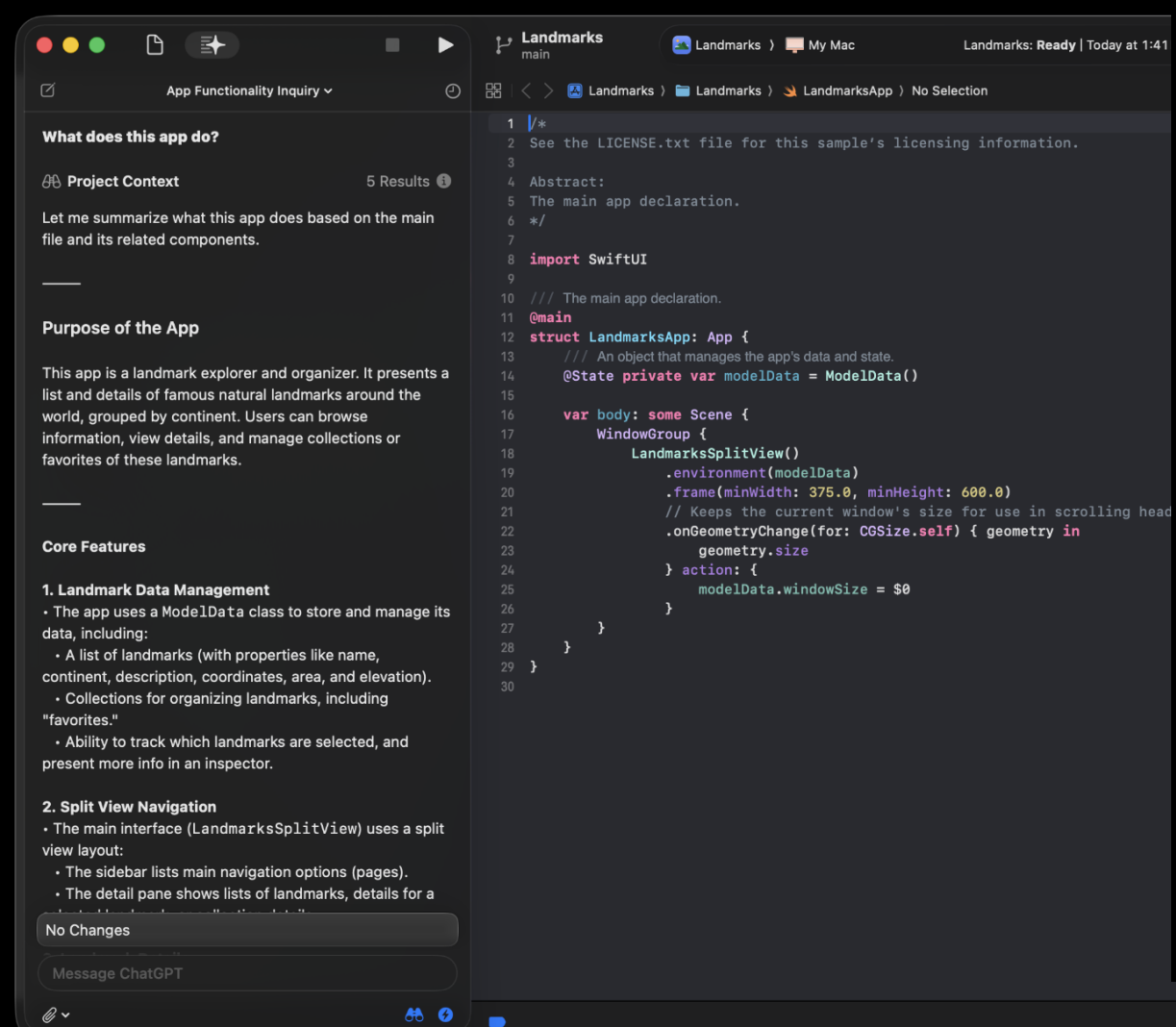
1. Pasta local onde será armazenado o projeto
2. Source Control
 - Criação de um repositório local para o projeto

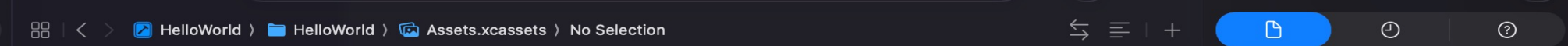
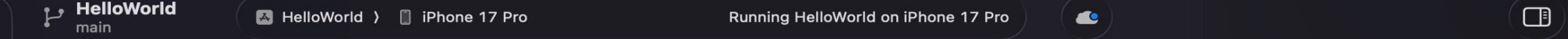
Xcode IDE | Intelligence

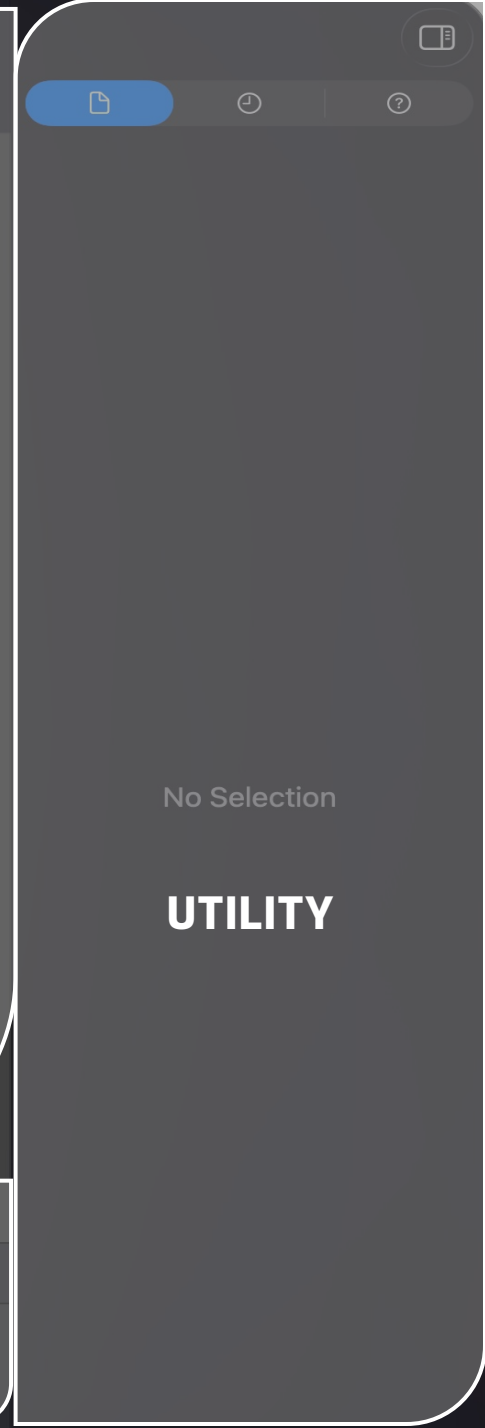
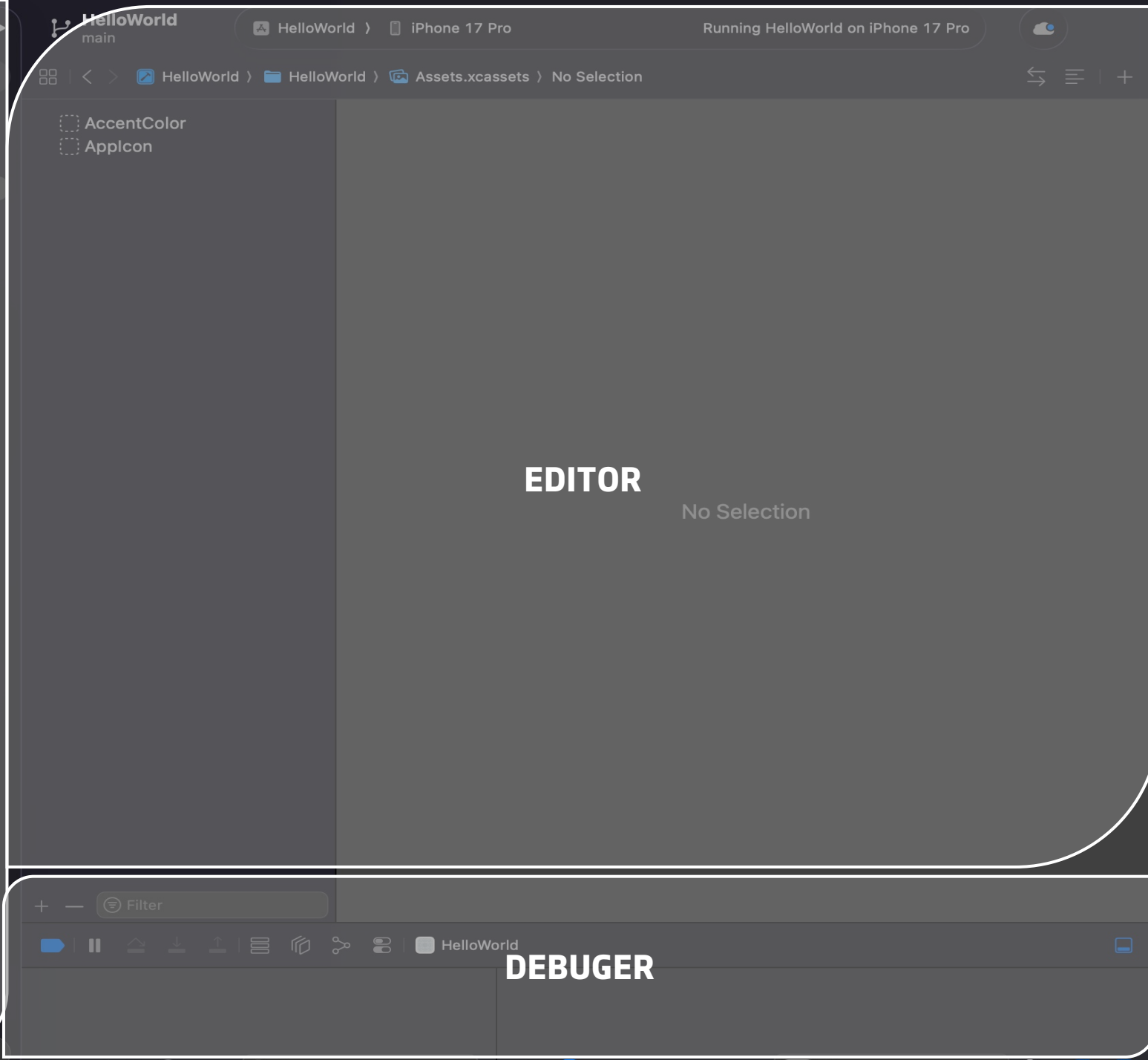
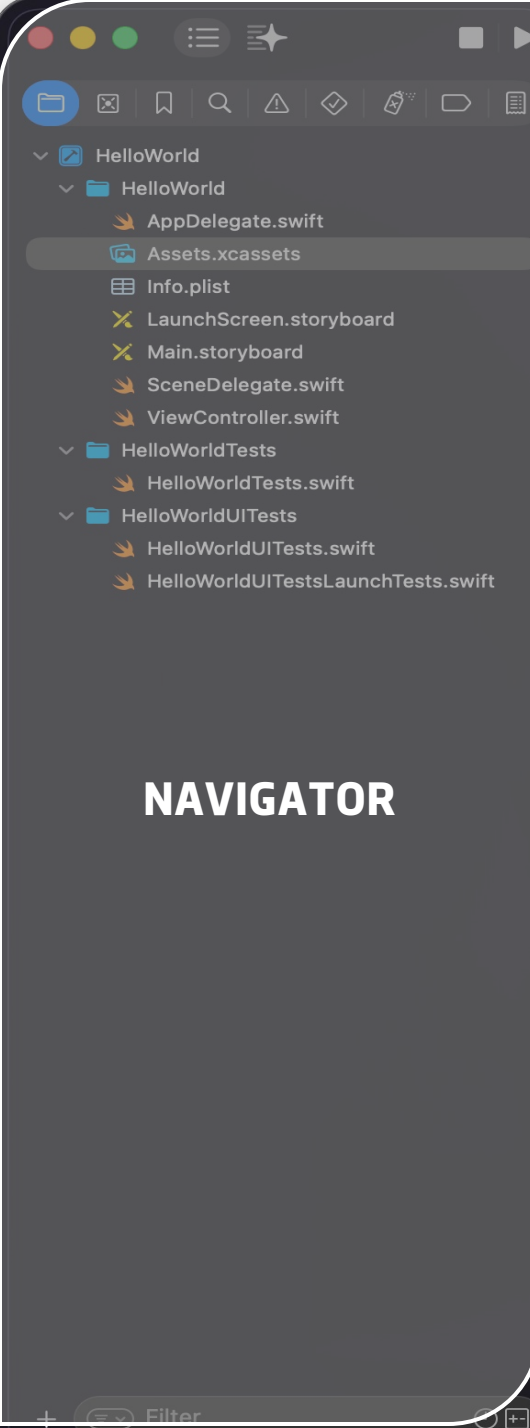


- Explore unfamiliar code
 - What does this app do?
- Learn about symbols and code
- Generate or modify code
 - Add properties and methods to a class
 - Create a list view and wrap it in a UINavigationController
 - Add the ability to edit the properties of items in the list view
 - Change the list view to a table view showing all the properties
- Apply changes to your code
- Customize the context of your prompts
- Generate playgrounds and previews
- Fix your code
- Generate documentation
- Browse previous conversations
- Rollback changes using the conversation history

Xcode IDE | Intelligence







Xcode IDE

Áreas de Trabalho

Navigator

- Acesso a ficheiros, símbolos, testes unitários, diagnósticos e outras características do projeto

Editor

- Principal área de desenvolvimento. Está sempre visível;
- Source Editor: permite escrever e editar código Swift.
- Interface Builder: criação e edição gráfica da user interface;
- Project Editor: visualização e edição de configurações como, por exemplo, build options.

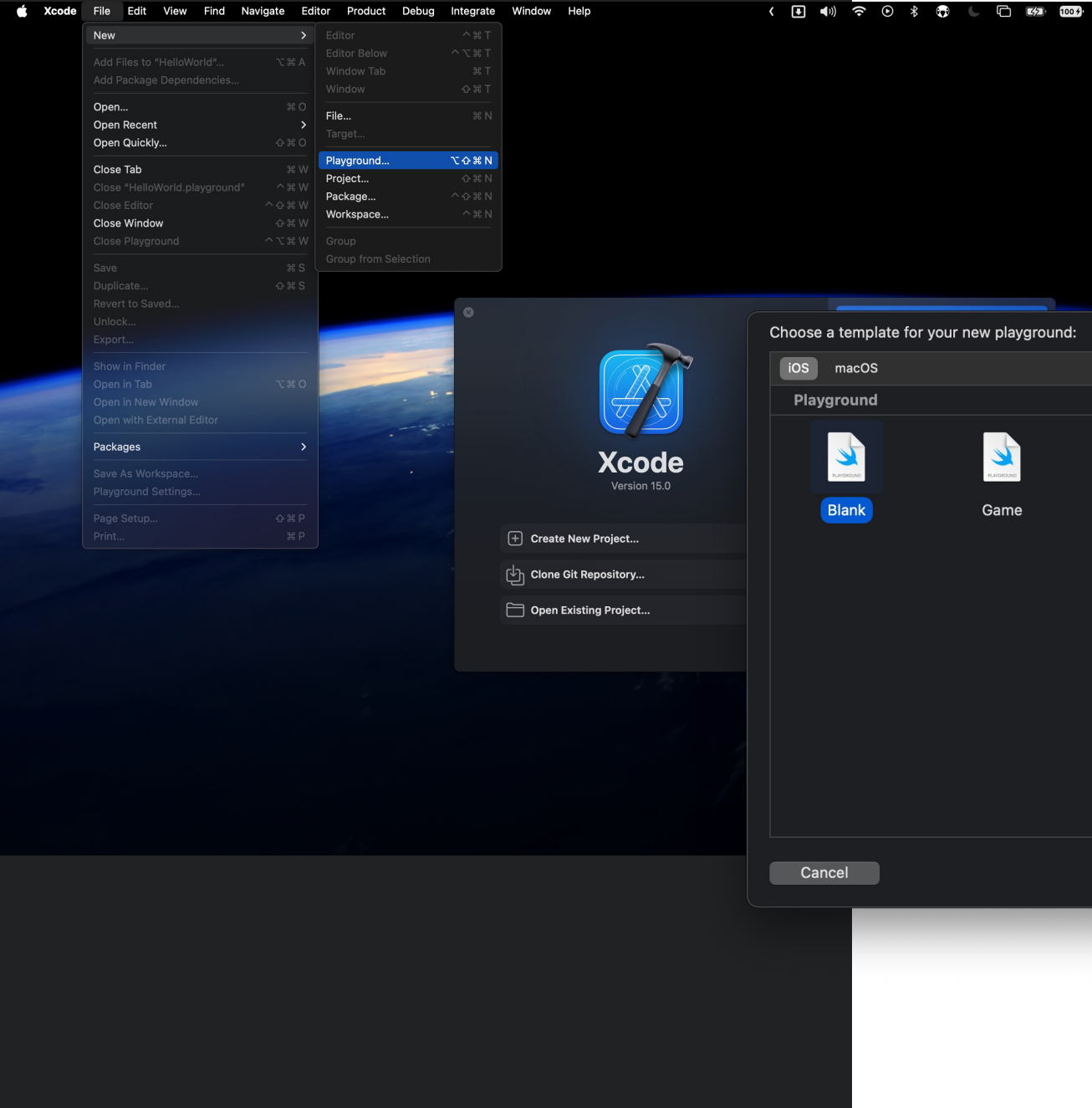
Playgrounds

- Permitem escrever código Swift de forma rápida;
- Resultado imediato após inserção de novas linhas de código;
- Visualização do progresso de execução numa *timeline*.

O *playground* permite:

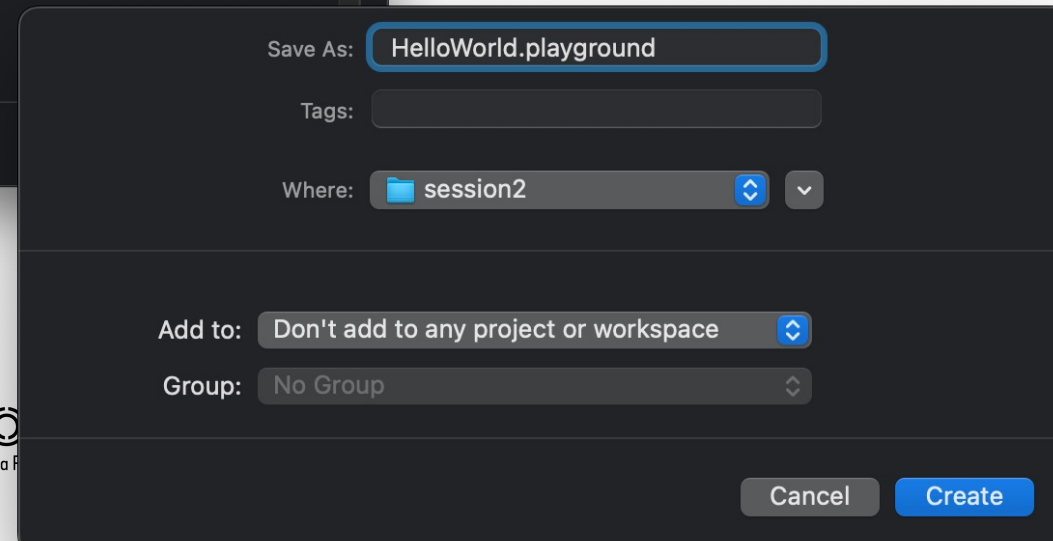
- Criar ou modificar um algoritmo e observar os resultados em cada passo;
- Criar novos testes e verificar que tudo está funcional.

Quando terminado o código é mais fácil movê-lo para um projeto Swift iOS



Hands on Swift Playgroung Program

1. Na barra de Menus:
 - Menu File – New – Playground
2. Selecionar Blank na tab iOS;
3. Colocar como nome HelloWorld
4. Gravar num local à escolha;

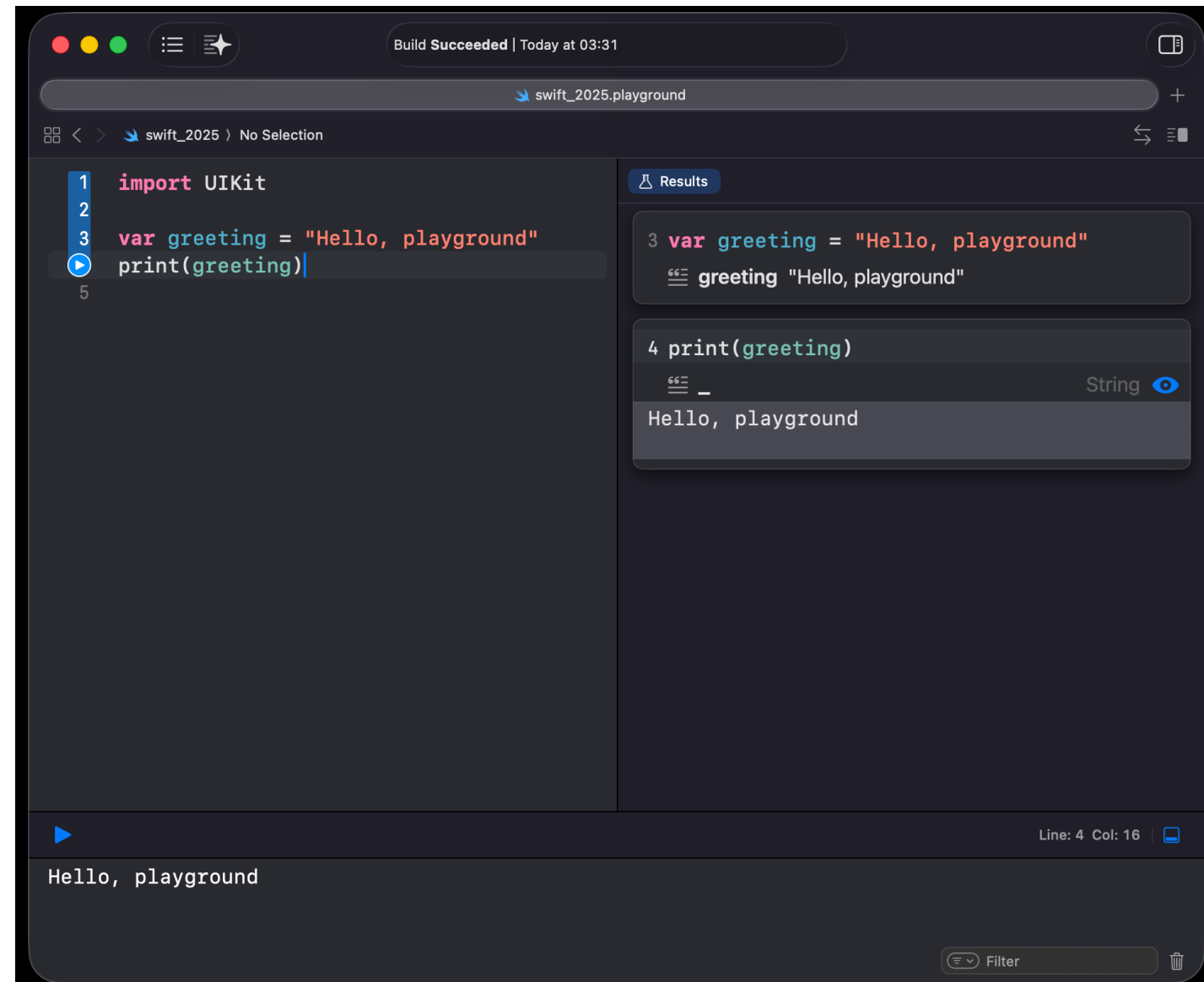


Hands on Swift Playgroung Program

O Xcode gera um ficheiro de playground com algum código pronto a utilizar.

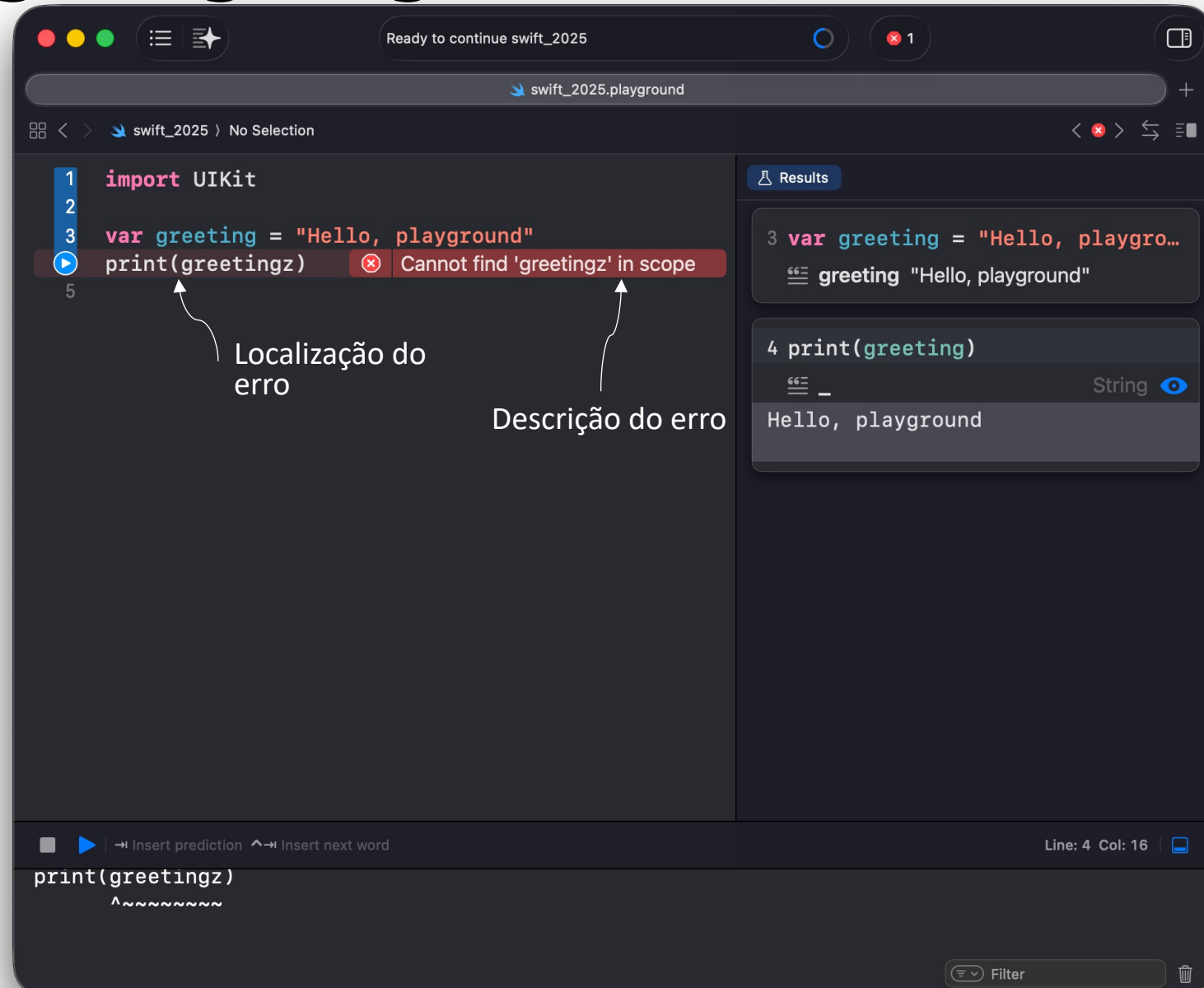
Características essenciais:

- Área de edição (Editor Area)
 - Local de escrita de código
 - Local para debug
- Área de resultados (Results Area)



Hands on: Swift Playground Program

1. Adicionar na linha 5 a seguinte instrução
`print(greeting)`
2. Provocar um erro de sintaxe
`print(greetingz)`





Swift

Swift

- A maioria das linguagens de programação executam as habituais tarefas que um computador necessita de executar como, por exemplo, guardar informação, comparar informação, tomar decisões sobre a informação armazenada, realizar ações de acordo com as decisões inferidas.
- O desenvolvimento do Swift iniciou-se em 2010 por Chris Lattner e em 9 de Setembro de 2014 foi oficialmente lançada a linguagem Swift 1.0
- O Swift possui um conjunto de ideias idênticas ao Objective-C, mas incorpora muitas funcionalidades de outras linguagens de programação modernas. Esta linguagem de programação foi desenvolvida de raiz para se tornar acessível ao programador comum.

Swift vs o resto

Existem dois tipos principais de linguagens de programação:

- Compiladas: Objective-C, C++, ...
 - São extremamente rígidas e requerem uma determinada sintaxe
 - Execução rápida
- Interpretadas: Ruby, PHP, Python, ...
 - Mais fáceis de aprender
 - Execução lenta

Swift vs o resto (2)

O Swift é uma linguagem que representa “o melhor de dois mundos”

- Incorpora a flexibilidade das linguagens interpretadas
- Possui a performance de linguagens compiladas, útil para jogos e aplicações exigentes

A Apple afirma que o Swift tem melhor desempenho do que aplicações implementadas em Objective-C

- Quase 4 vezes mais rápido que Python
- 40% mais rápido que Objective-C

Tipos de Dados em Swift

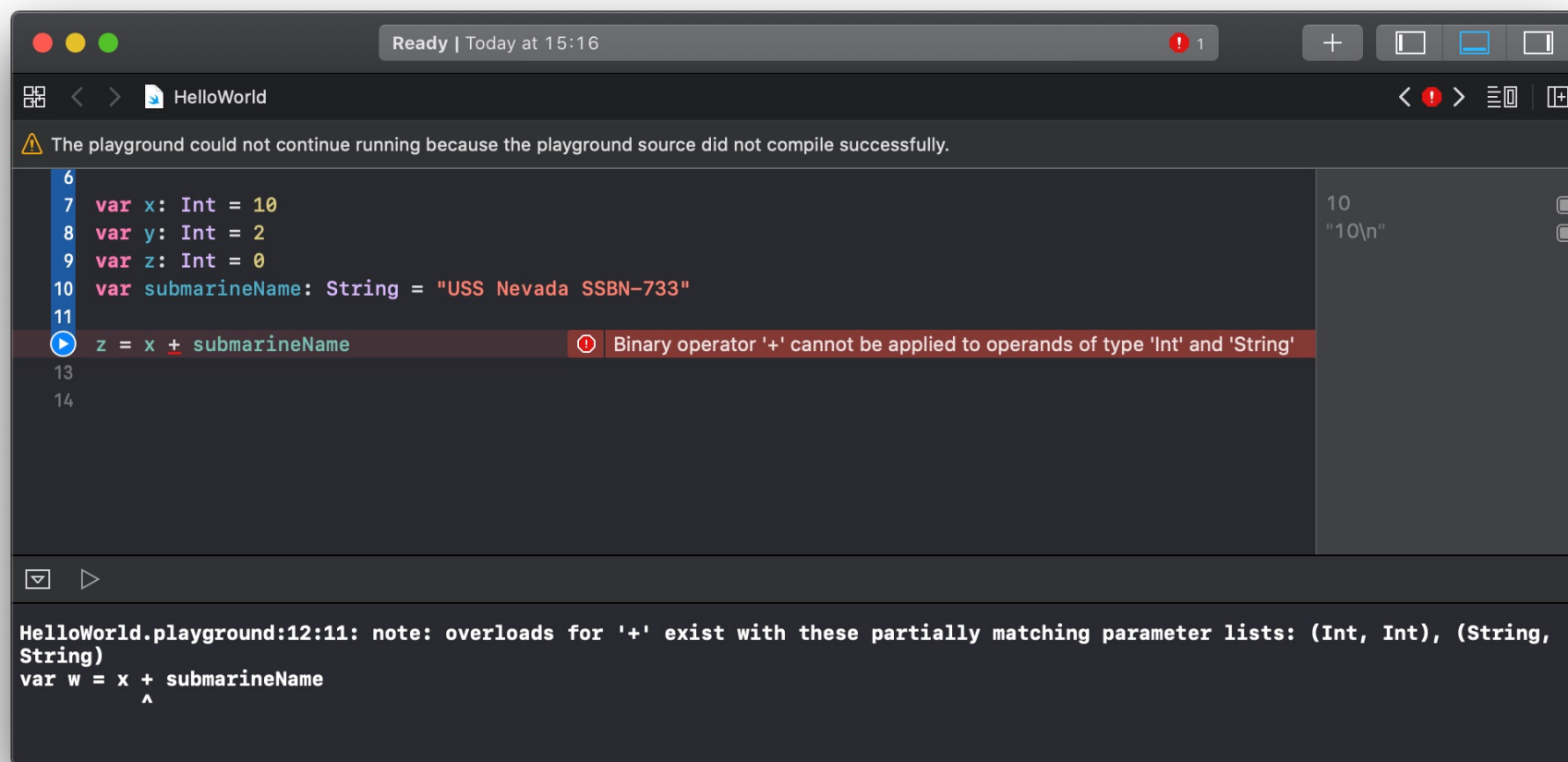
Exemplos de declaração de variáveis com tipo de dados específico

- `var x: Int = 10`
- `var y: Int = 2`
- `var z: Int = 0`
- `var submarineName: String = "USS Nevada SSBN-733"`

Type	Examples
Int	1, 5, 10, 100
Float or Double	1.0, 2.222, 3.14159
Bool	true, false
String	"Star Wars", "Star Trek"
ClassName	UIView, UILabel, and so on

Tipos de Dados em Swift (2)

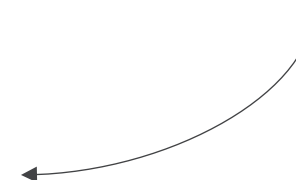
Tal como noutras linguagens, não podemos misturar tipos de dados...



Declaração de variáveis

```
var <nome>:<tipo de dados> = <valor>  
var firstNumber: Int = 5
```

Declaração explícita



```
var <nome> = <valor>  
var firstNumber = 5
```

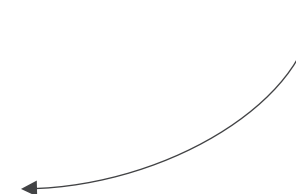
Declaração implícita
(mais frequente, e recomendada pela documentação)



Declaração de constantes

```
let <nome>:<tipo de dados> = <valor>  
let firstNumber: Int = 5
```

Declaração explícita



```
let <nome> = <valor>  
let firstNumber = 5
```

Declaração implícita
(mais frequente, e recomendada pela
documentação)



Valor Boleano e operadores

Apenas contém valores True ou False

```
var seeMovies: Bool = false
```

Operator	Definition
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Exactly equal to
!=	Not equal to

Objetos

Swift incorpora o melhor do Objective-C sem a restrição de ser obrigatoriamente compatível com C e inclui as melhores características de linguagens interpretadas.

- Características que tornam o Swift uma POO;
- Quase tudo é um objeto;
- Objetos contêm variáveis de instância;
- Objetos e variáveis de instância têm um âmbito definido;
- Classes escondem a implementação do objeto.

Classe em Swift

```
9  import UIKit
10
11  class HelloWorld {
12      func logMessage() {
13          let helloWorld: String = "Hello World!"
14          print(helloWorld)
15      }
16  }
```

HelloWorld representa uma classe

- Se quisermos que HelloWorld seja uma subclasse (ou seja, estenda outra classe) da classe LogFile a declaração seria
class HelloWorld: LogFile {...}
- Criação/utilização do objeto

```
let myHelloWorld = HelloWorld()
myHelloWorld.logMessage()
```

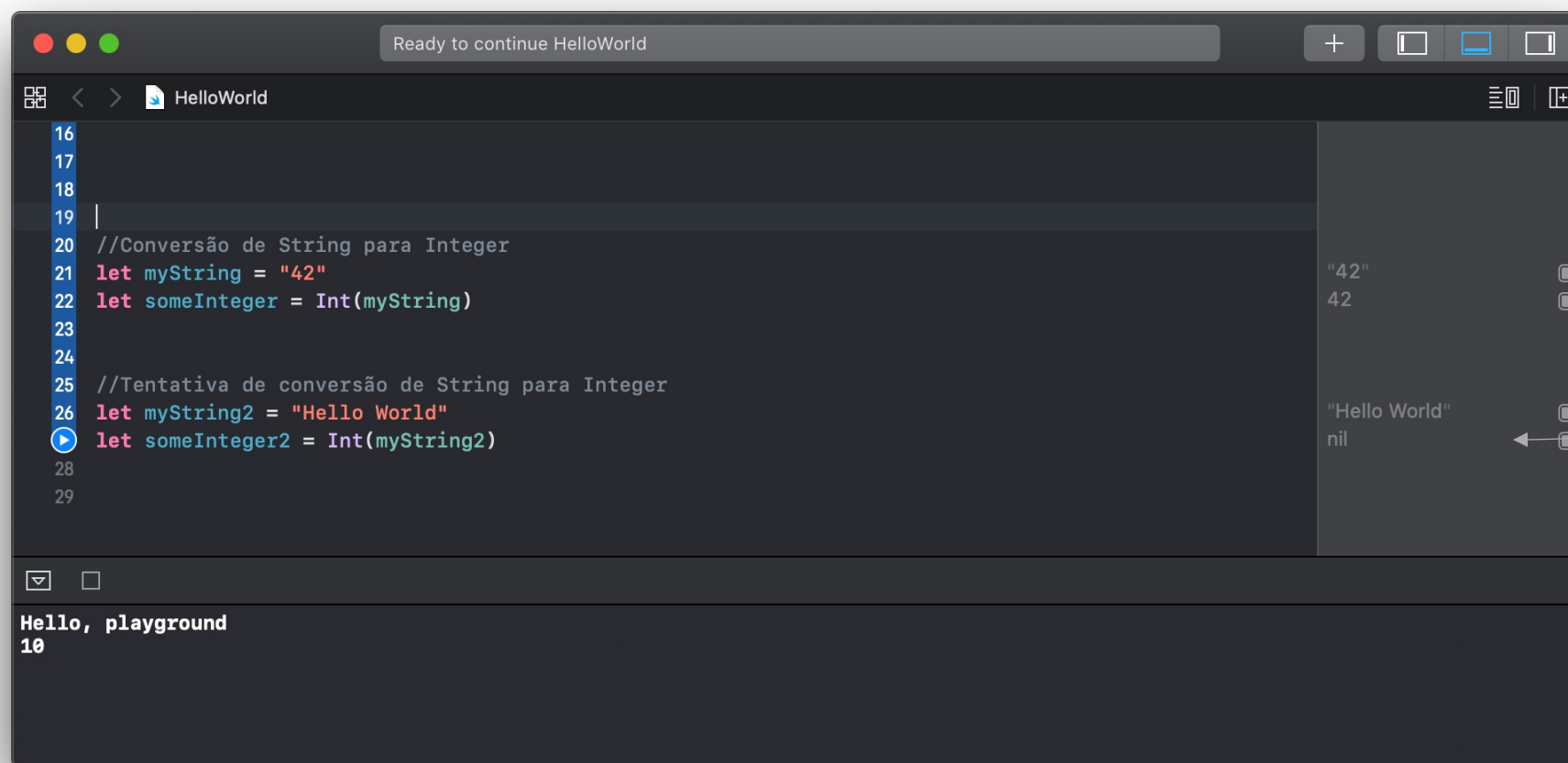

Classe em Swift

- Os métodos dentro de uma classe podem ser
 - Método da classe (equivalente ao static em Java);
 - Método da instância (semelhante aos métodos private, public e protected em Java).
- No caso do objeto HelloWorld o método implementado é um método de instância
 - Apenas pode ser utilizado após a criação do objeto.
- Para o tornar num método de classe
 - Incluir a palavra class antes da função (func);
 - Permite a utilização do método antes da criação do objeto, porém não se tem acesso às propriedades do objeto;
 - Exemplo do método logMessage como método de classe

```
class func logMessage(){...}
```

Optionals (opcionais)

Utilizados quando um valor pode estar “ausente”, ou seja, uma variável pode ou não ter um valor associado.



```
16
17
18
19
20 //Conversão de String para Integer
21 let myString = "42"
22 let someInteger = Int(myString)
23
24
25 //Tentativa de conversão de String para Integer
26 let myString2 = "Hello World"
27 let someInteger2 = Int(myString2)
28
29
```

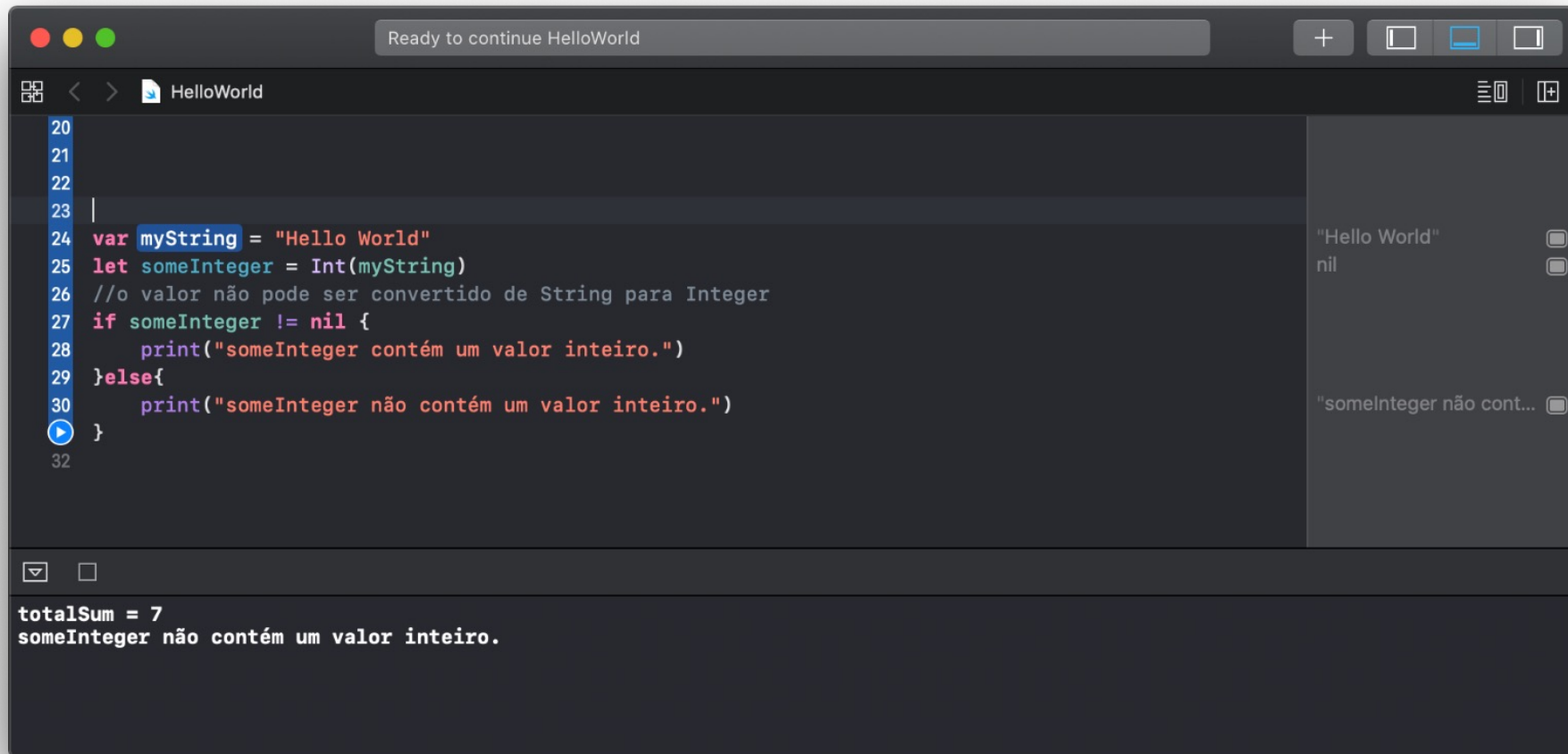
The IDE interface shows a file named 'HelloWorld'. The code is in a dark-themed editor. Line 27 is highlighted with a blue cursor. To the right of the code, there is a variable inspector showing the values of 'myString' (42) and 'someInteger2' (nil). Below the code editor, there is a console output area showing 'Hello, playground' and '10'.

Optional pois não é possível realizar a conversão.

O IDE assume como valor nulo (ou nil)

Optionals (opcionais)

Como pode não conter valor é necessário verificar antes de aceder ao seu conteúdo!

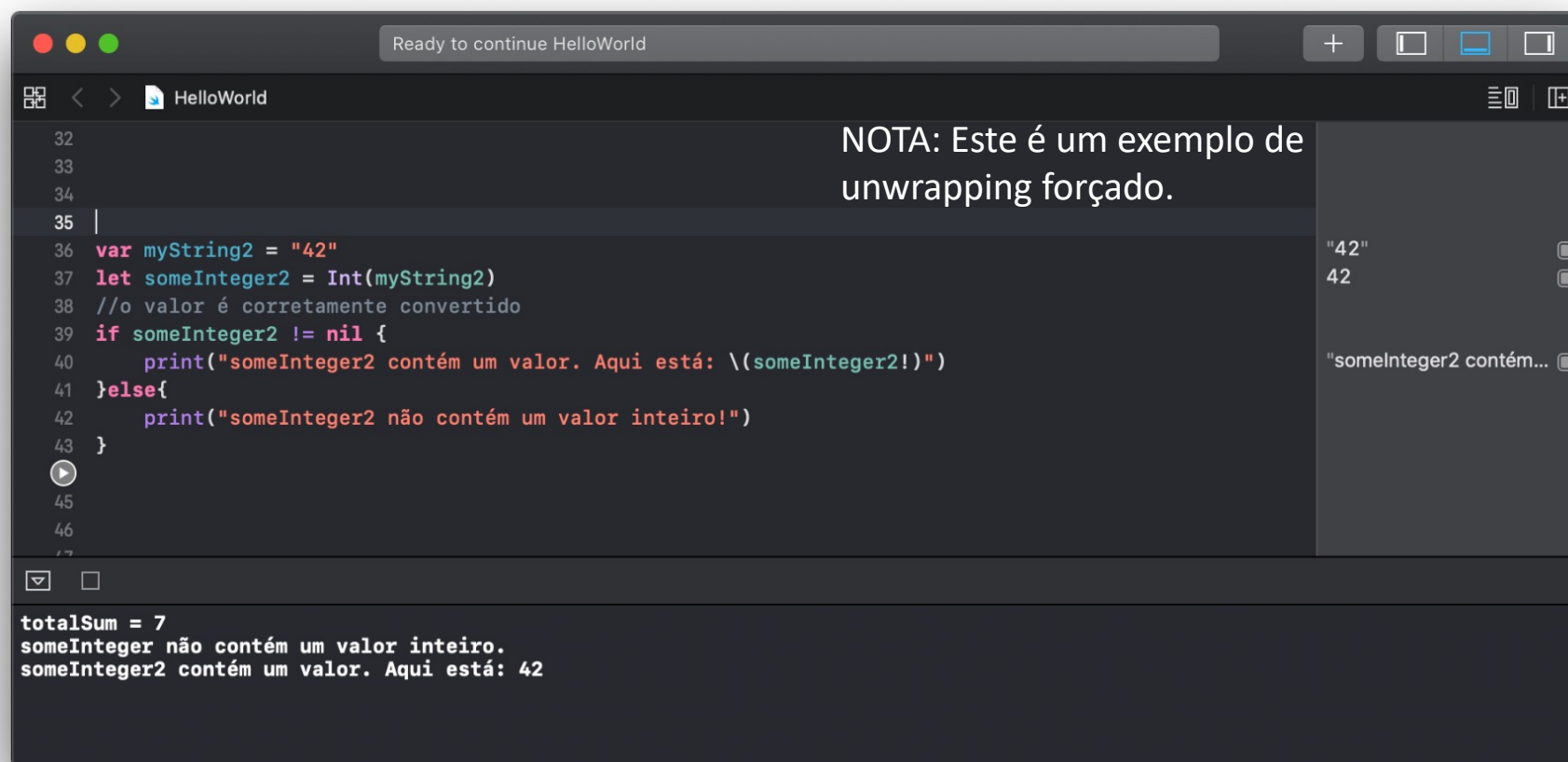


```
20
21
22
23 |
24 var myString = "Hello World"
25 let someInteger = Int(myString)
26 //o valor não pode ser convertido de String para Integer
27 if someInteger != nil {
28     print("someInteger contém um valor inteiro.")
29 }else{
30     print("someInteger não contém um valor inteiro.")
31 }
32
```

totalSum = 7
someInteger não contém um valor inteiro.

Optionals (opcionais)

Após validação o acesso ao conteúdo é realizado através do ponto de exclamação (!) após o nome do opcional. O '!' indica que foi realizada a verificação e que é seguro aceder ao conteúdo.



```
32
33
34
35 |
36 var myString2 = "42"
37 let someInteger2 = Int(myString2)
38 //o valor é corretamente convertido
39 if someInteger2 != nil {
40     print("someInteger2 contém um valor. Aqui está: \(someInteger2!)")
41 }else{
42     print("someInteger2 não contém um valor inteiro!")
43 }
44
45
46
```

NOTA: Este é um exemplo de unwrapping forçado.

"42" ☐

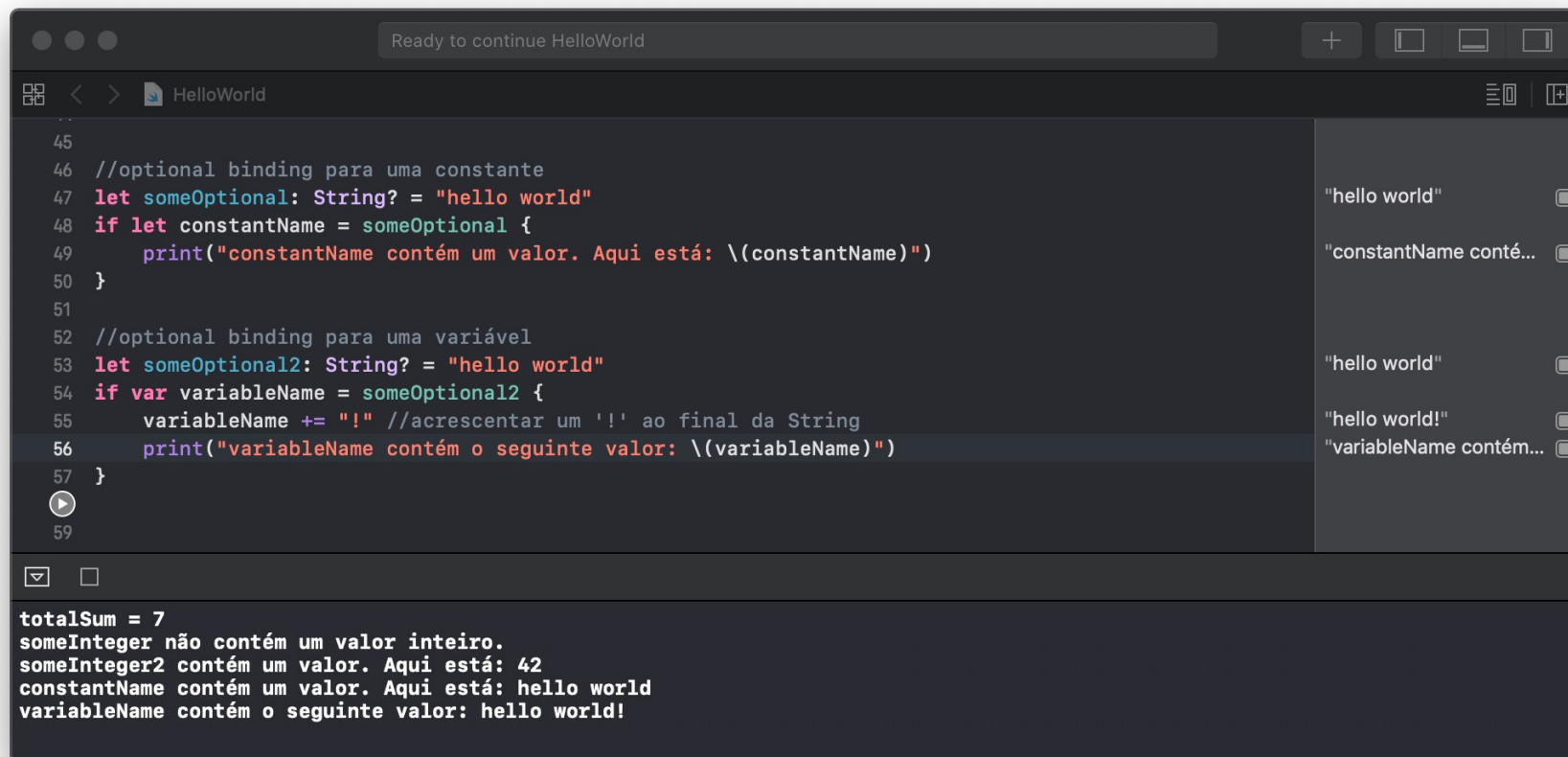
42 ☐

"someInteger2 contém..." ☐

totalSum = 7
someInteger não contém um valor inteiro.
someInteger2 contém um valor. Aqui está: 42

Optional Binding | **if let**

É possível perceber se um *optional* contém valor e, em caso afirmativo, atribuí-lo temporariamente a uma variável ou constante para executar uma determinada ação. Este processo (designado *optional binding*) pode ser utilizado na condição do *if* ou *while* para determinar se o *optional* contém valor.



```
45
46 //optional binding para uma constante
47 let someOptional: String? = "hello world"
48 if let constantName = someOptional {
49     print("constantName contém um valor. Aqui está: \(constantName)")
50 }
51
52 //optional binding para uma variável
53 let someOptional2: String? = "hello world"
54 if var variableName = someOptional2 {
55     variableName += "!" //acrescentar um '!' ao final da String
56     print("variableName contém o seguinte valor: \(variableName)")
57 }
58
59
```

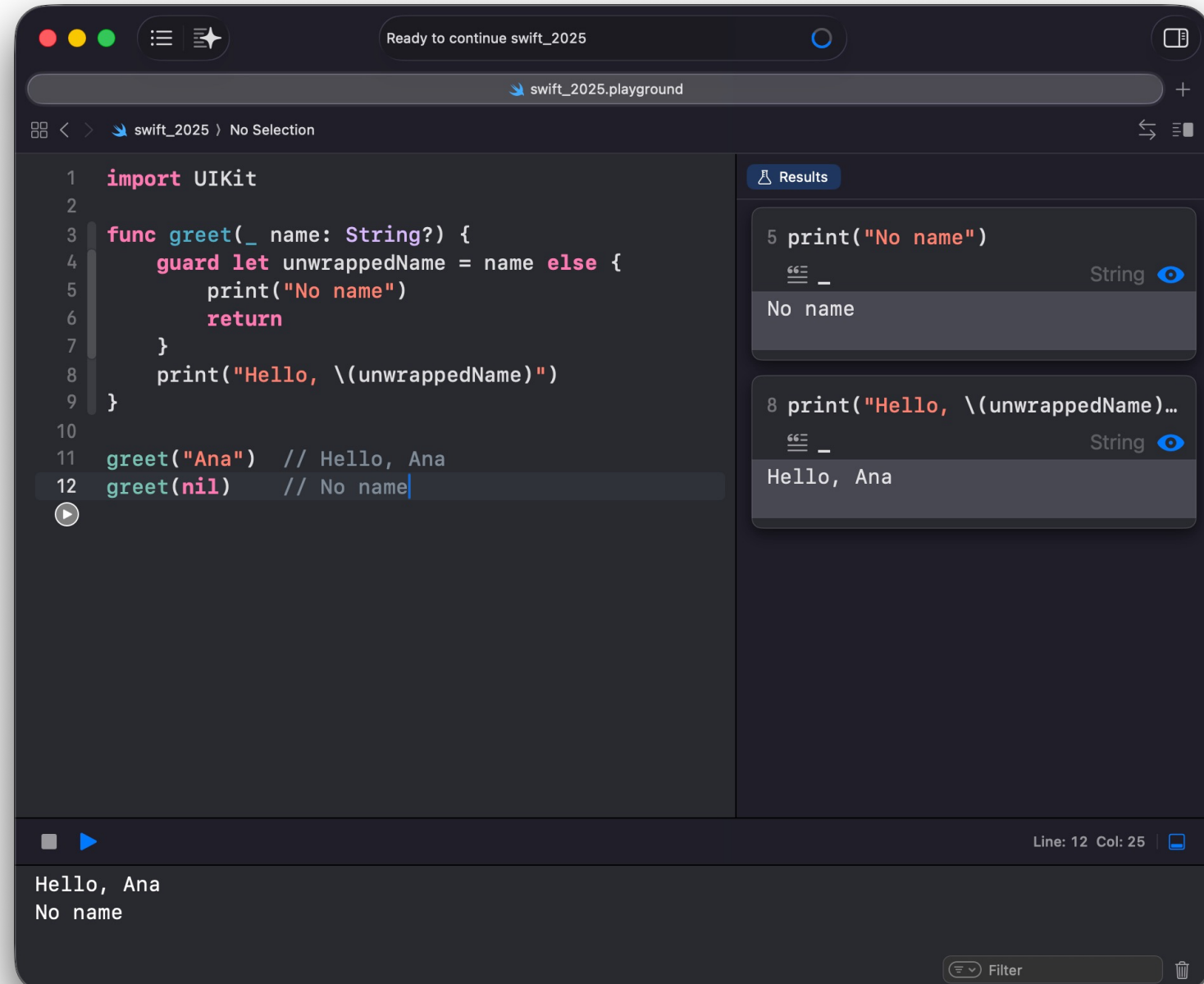
totalSum = 7
someInteger não contém um valor inteiro.
someInteger2 contém um valor. Aqui está: 42
constantName contém um valor. Aqui está: hello world
variableName contém o seguinte valor: hello world!

NOTA:

Neste caso não é necessário utilizar o `!` nem verificar se a variável possui conteúdo. Caso a conversão seja realizada com sucesso a variável/constante pode ser utilizada diretamente.

Optional Binding | **guard let**

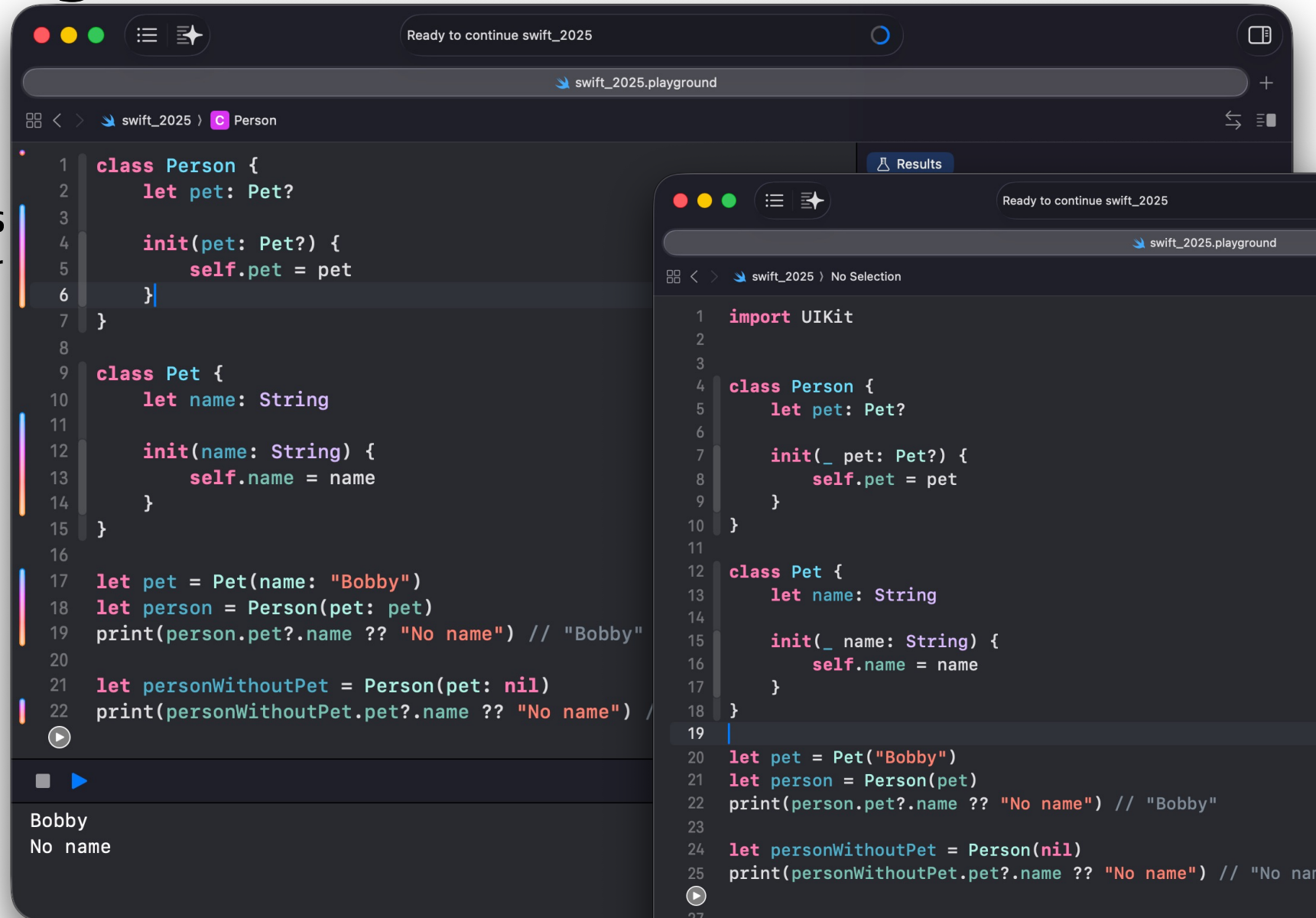
Usado normalmente em funções, retorna se o valor for nil.



Optional Chaining | ?.

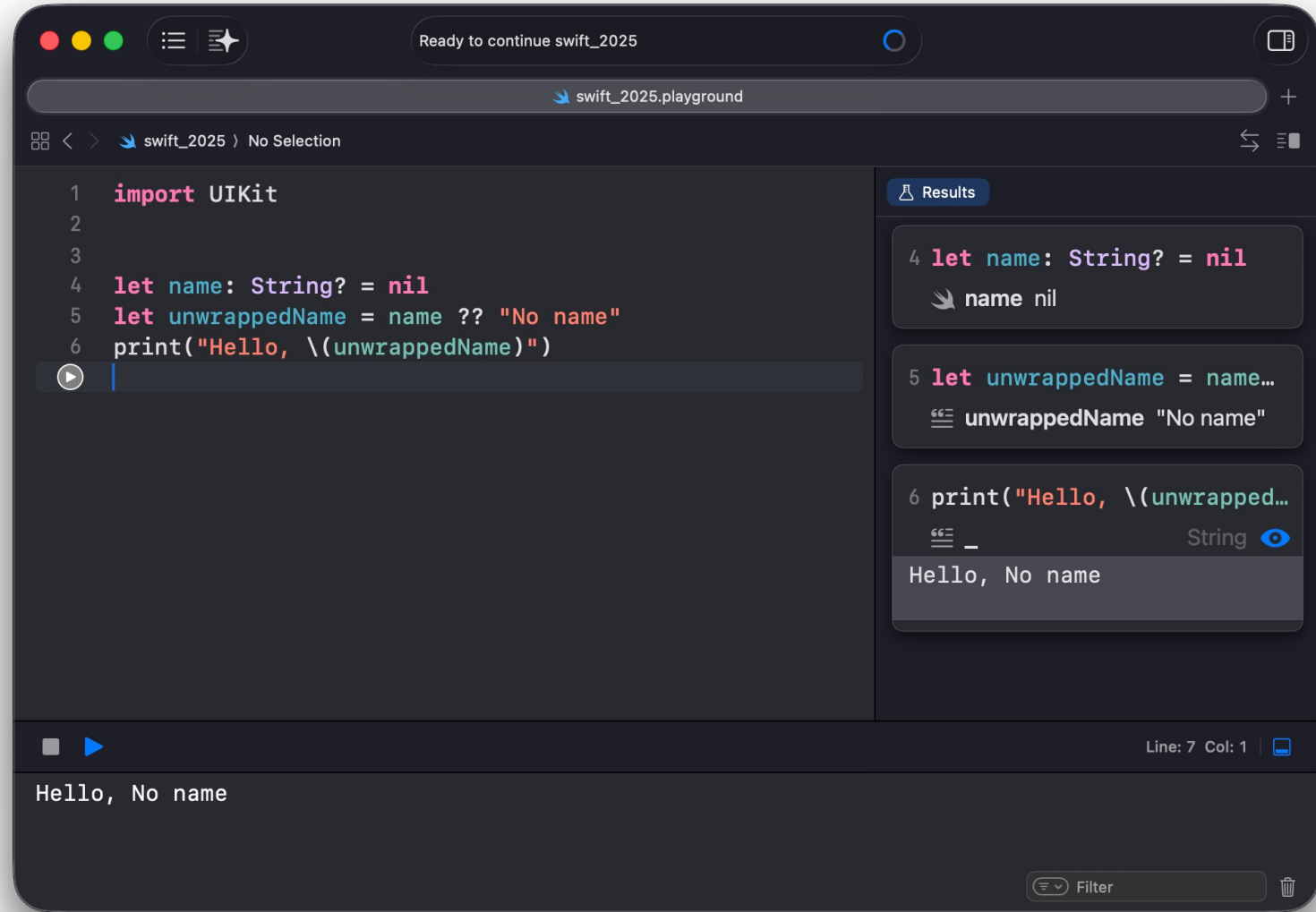
Permite chamar propriedades ou métodos apenas se o valor não for nil.

Retornando propriedades, deve-se definir um valor por omissão



Nil-Coalescing Operator | ??

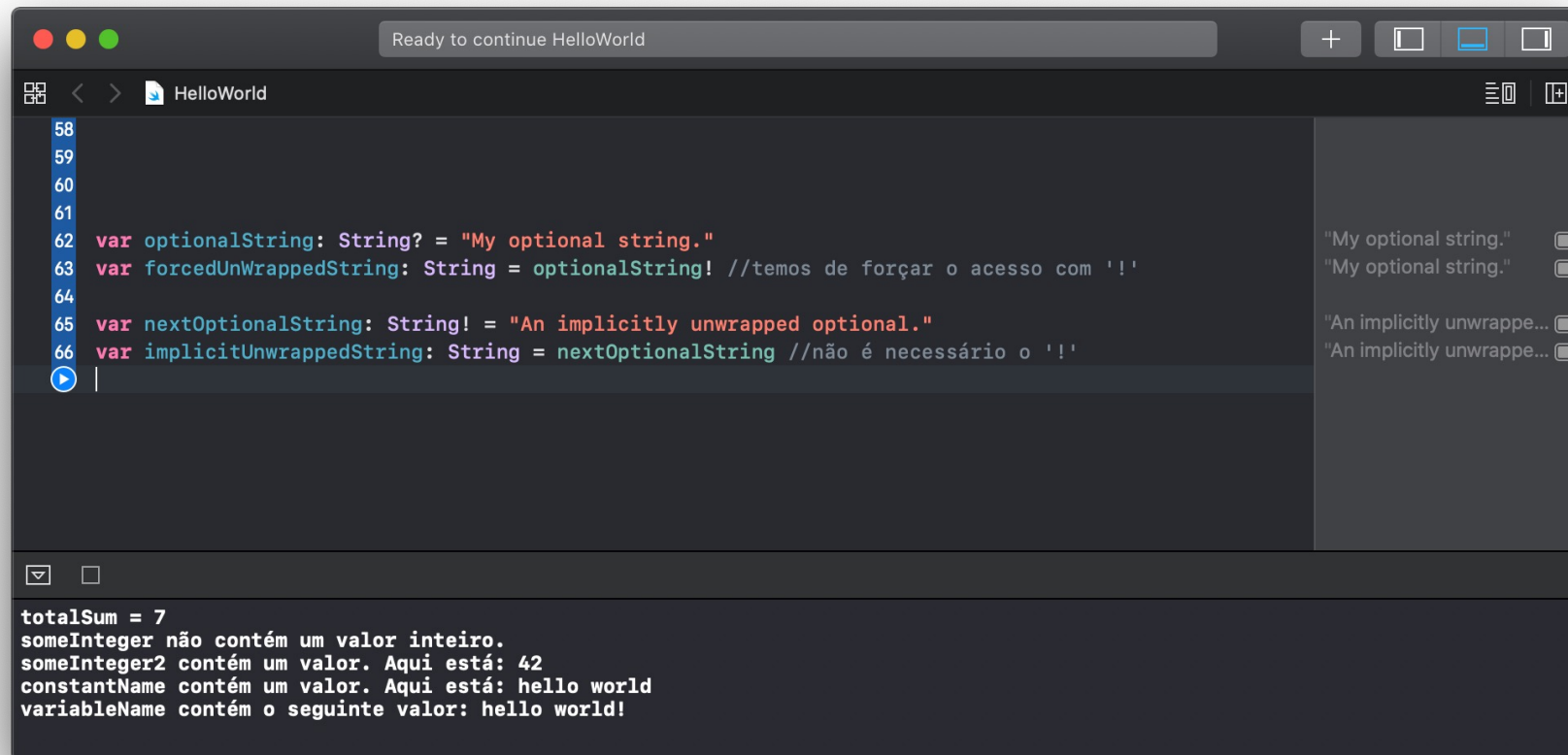
Define um valor por omissão se for nil.



Optional Binding (implícito)

Pode ser útil garantir que um *optional* tenha sempre um valor atribuído, deixando de ser necessário verificar se é diferente de nil e adicionar o '!' para aceder ao valor.

Estes *optionals* são designados *implicitly unwrapped optionals*



```
58
59
60
61
62 var optionalString: String? = "My optional string."
63 var forcedUnwrappedString: String = optionalString! //temos de forçar o acesso com '!'
64
65 var nextOptionalString: String! = "An implicitly unwrapped optional."
66 var implicitUnwrappedString: String = nextOptionalString //não é necessário o '!'

" My optional string."
" My optional string."
" An implicitly unwrapped optional."
" An implicitly unwrapped optional."

totalSum = 7
someInteger não contém um valor inteiro.
someInteger2 contém um valor. Aqui está: 42
constantName contém um valor. Aqui está: hello world
variableName contém o seguinte valor: hello world!
```

Ciclos

Para o desenvolvimento da aplicação são necessário ciclos!

Ciclos count-controlled (*for*)

- Executam um determinado número de vezes

Ciclos condition-controlled (*while*)

- Executam enquanto uma condição for válida

Ciclos infinitos

- Executam sempre

```
//Count-controlled loop
for i in 0..<10 {
    print("O índice é: \(i)")
}

for i in 1...10 {
    print("\(i) vezes 10 é \(i * 10)")
}
```

```
//Infinite loop
var x = 0
while x != 5{
    x += 2
}

while true {
    //executa o código para sempre...
}
```

```
//Condition-controlled loop
var isTrue = true
while isTrue {
    //realiza uma operação
    isTrue = false //uma determinada condição ocorre e altera o valor de isTrue para false
}
```

Flowchart

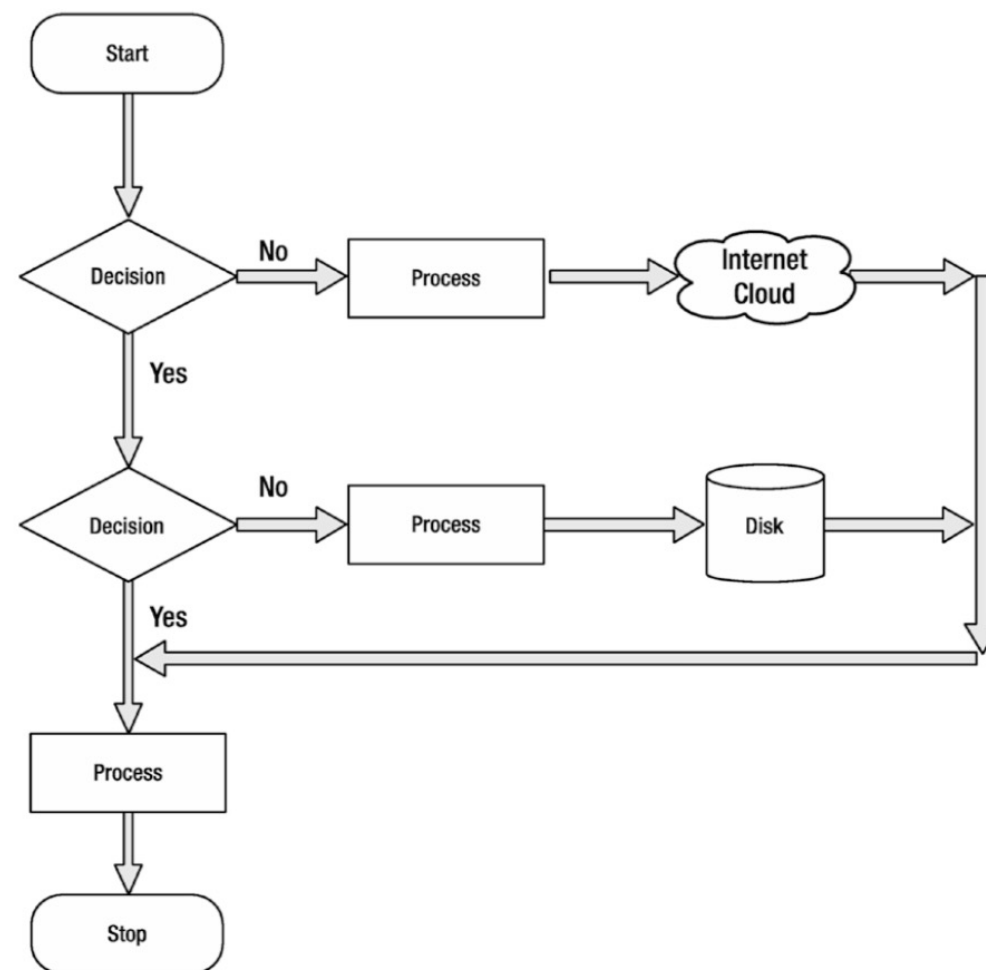
Algoritmia com diagramas

Representação de um algoritmo através de diferentes tipos de caixas conectadas por linhas e setas.

Utilizado para visualmente mostrar a execução do código.

Têm sempre um início (*start*) e fim (*end*).

Os ramos (*branches*) nunca devem terminar sem chegar ao *stop*. Ajuda a garantir que todos os *branches* foram contabilizados e que terminam corretamente a sua execução.

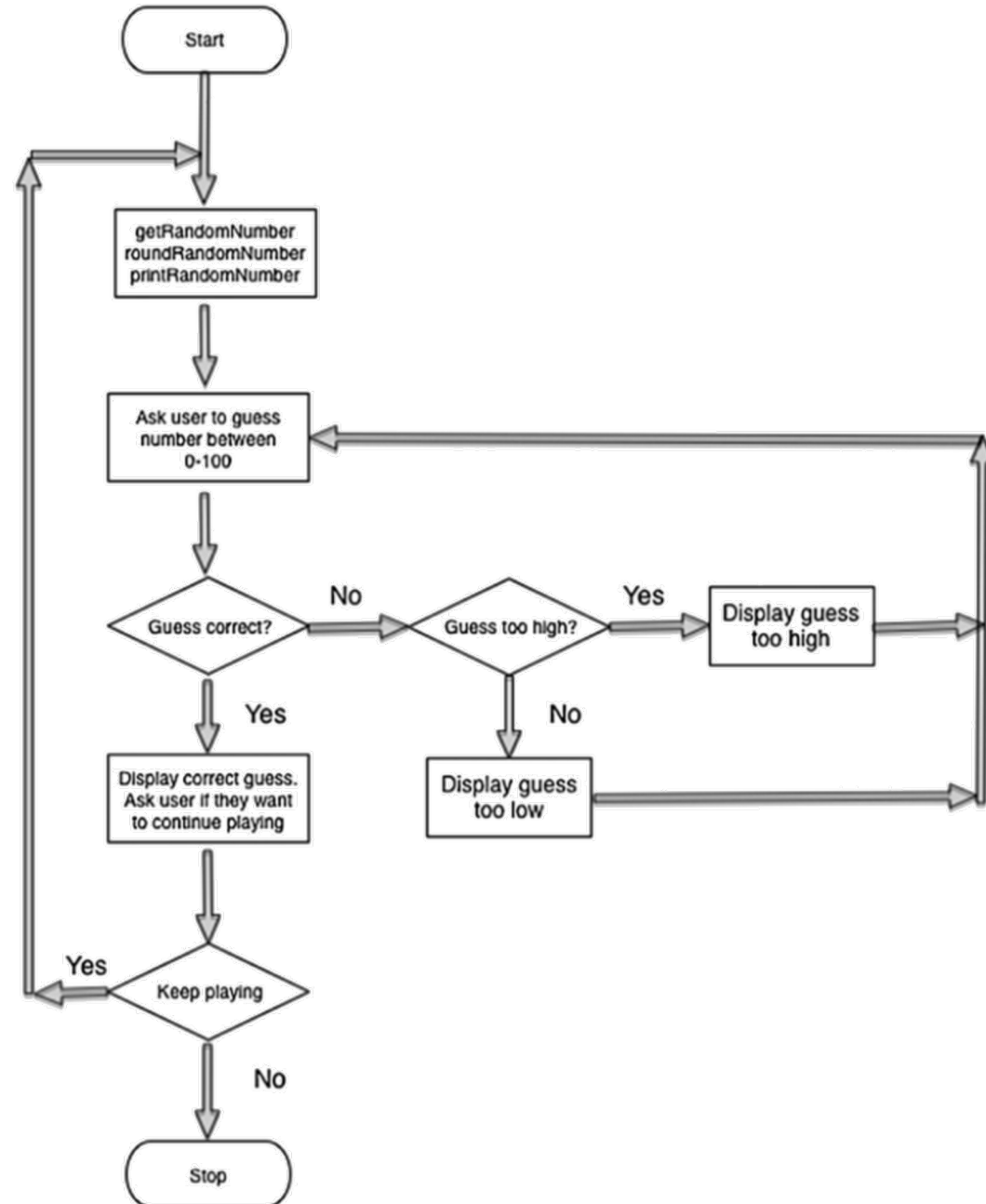


Hands on

Desenvolvimento de uma app

Pretende-se o desenvolvimento de uma aplicação que gera um número aleatório entre 0 e 100 e pede ao utilizador que adivinhe o número.

O utilizador tem de realizar várias tentativas até acertar. Quando der a resposta correta, a aplicação questiona se o utilizador pretende jogar novamente.



Hands on

Desenvolvimento de uma app

O playground não permite capturar input do utilizador. Vamos criar uma Mac Console App!

1. Selecionar a criação de um novo projeto Xcode;
2. Selecionar *Command Line Tool* na tab macOS;
3. Colocar como nome RandomNumber e garantir que a linguagem seleciona é Swift;
4. No main.swift escrever o código necessário.



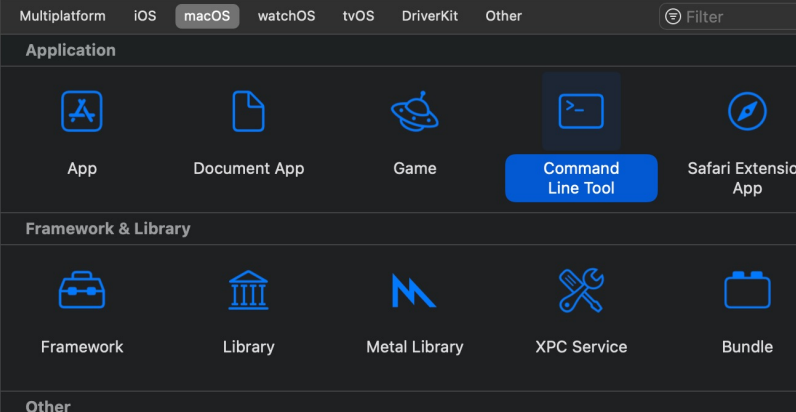
Xcode
Version 15.0

+ Create New Project...

↓ Clone Git Repository...

📁 Open Existing Project...

Choose a template for your new project:



```
import Foundation
```

```
var randomNumber = 1  
var continueGuessing = true  
var keepPlaying = true  
var input = ""
```

Função que retorna um número aleatório entre 0 e um limite definido como argumento

```
while keepPlaying {  
    //obter um número aleatório entre 0 e 100  
    randomNumber = Int(arc4random_uniform(101))  
    print("O número aleatório para adivinhar é: \(randomNumber)")  
  
    while continueGuessing {  
        print("Introduza um número entre 0 e 100.")  
        //obter o input do teclado  
        input = String(bytes: FileHandle.standardInput.availableData, encoding: .utf8)!  
        input = input.trimmingCharacters(in: .whitespacesAndNewlines)  
  
        if let userGuess = Int(input) {  
            if userGuess == randomNumber {  
                continueGuessing = false  
                print("Número correto!")  
            } else if userGuess > randomNumber {  
                //o utilizador inseriu um número maior  
                print("Número introduzido é maior que o gerado!")  
            } else {  
                print("Número introduzido é menor que o gerado!")  
            }  
        }  
    }  
}
```

Converter o input lido para inteiro. Pode não ser possível (optional binding)

```
    }else{
        print("Valor inválido, tente de novo.")
    }|
}

print("Jogar de novo? S ou N")
input = String(bytes: FileHandle.standardInput.availableData, encoding: .utf8)!
input = input.trimmingCharacters(in: .whitespacesAndNewlines)

if input == "N" || input == "n"{
    keepPlaying = false
}

continueGuessing = true
}
```

Remove caracteres extra como
espaços e '\n' (nova linha)

P00 – Criação de Objetos em Swift

1. Abrir o Xcode e seleccionar File > New > Project
2. Seleccionar iOS (topo do menu) e aplicação do tipo Master-Detail
3. Introduzir BookStore como nome do projeto e pt.ipp.estg como company
4. Seleccionar a pasta BookStore no navegador do projeto (Project Navigator)
5. Seleccionar File > New > File
6. Criar um ficheiro do tipo Cocoa Touch Class (tab iOS, secção Source)
7. Inserir como nome da classe Customer que tem como super classe NSObject
8. Abrir o novo ficheiro Customer.swift e iniciar a introdução dos dados

Customer
First Name
Last Name
Address Line 1
Address Line 2
City
State
Zip
Phone Number
Email Address
Favorite Book Genre
List Purchase History

Objeto Customer

```
class Customer: NSObject {  
    var firstName = ""  
    var lastName = ""  
    var addressLine1 = ""  
    var addressLine2 = ""  
    var city = ""  
    var state = ""  
    var zip = ""  
    var phoneNumber = ""  
    var emailAddress = ""  
    var favoriteGenre = ""  
  
    func listPurchaseHistory() -> [String]{  
        return ["Purchase 1", "Purchase 2"]  
    }  
}
```

Propriedades do objetos
definidas como variáveis

Função (método) que não recebe argumentos
e retorna um array de String.

NOTA: este método é estático e apenas
demonstrativo!

P00

Porquê utilizar?

Tal como em Java, o Swift utiliza P00 uma vez que permite

Herança (Inheritance)

- Nota: em Swift não existe uma superclass a não ser que seja especificamente definida tal como acontece no exemplo anterior cuja superclass é do tipo NSObject

Interface

Polimorfismo

- Capacidade de um objeto de uma classe ser utilizado como objeto noutra classe

Value Orientated Programming

Novo paradigma introduzido pela Apple para os programadores em iOS.

É recomendada a utilização de Structs em detrimento de Class para excertos simples de dados.

A Struct é similar a uma Class, mas é passado por valor para um método e **não** pode herdar informação da superclass. Leva a um menor esforço computacional no momento da criação da estrutura.

Uma Struct é instanciada e utilizada da mesma forma que uma Class.

```
struct Customer {  
    var firstName = ""  
    var lastName = ""  
    var addressLine1 = ""  
    var addressLine2 = ""  
    var city = ""  
    var state = ""  
    var zip = ""  
    var phoneNumber = ""  
    var emailAddress = ""  
    var favoriteGenre = ""  
  
    func listPurchaseHistory() -> [String]{  
        return ["Purchase 1", "Purchase 2"]  
    }  
}
```

Atenção: uma Struct deve ser utilizada quando não é necessário herança. Porém podem existir outros fatores na decisão...

Class (by reference) | Struct (by value)

Ready to continue swift_2025

swift_2025.playground

```
1 import UIKit
2
3
4 class Person {
5     var name: String?
6
7     init(name: String?) {
8         self.name = name
9     }
10 }
11
12 var person = Person(name: "John")
13
14 // copied person into cloned person
15 var clonedPerson = person
16 clonedPerson.name = "Rambo"
17
18 print(person.name ?? "No name")
19 print(clonedPerson.name ?? "No name")
```

Results

- 12 var person = Person(name: "John")
- 15 var clonedPerson = person
- 16 clonedPerson.name = "Rambo"
- 18 print(person.name ?? "No name")
- 19 print(clonedPerson.name ?? "No name")

Rambo
Rambo

Same names -> both variables point to the same object!

Ready to continue swift_2025

swift_2025.playground

```
1 import UIKit
2
3
4 struct Person {
5     var name: String?
6 }
7
8 var person = Person(name: "John")
9
10 // copied person into cloned person
11 var clonedPerson = person
12 clonedPerson.name = "Rambo"
13
14 print(person.name ?? "No name")
15 print(clonedPerson.name ?? "No name")
```

Results

- 8 var person = Person(name: "John")
- 11 var clonedPerson = person
- 12 clonedPerson.name = "Rambo"
- 14 print(person.name ?? "No name")
- 15 print(clonedPerson.name ?? "No name")

John
Rambo

Different names -> variables point to the distinct objects!

Line: 5 Col: 22

Filter

Refactoring

Processo de examinar o código e reescrevê-lo para o tornar

- mais eficiente;
- mais perceptível;
- mais fácil de manter.

“As developers, we have found that the best line of code is the line that you don’t have to write – less code means less to debug and maintain.”

(Como programadores sabemos que a melhor linha de código é aquela que não é necessário escrever – menos código significa menos para debug e manutenção.)

Kaczmarek, S., Lees, B., & Bennett, G. (2019). Swift 5 for Absolute Beginners: Learn to Develop Apps for iOS.

Leitura recomendada

- **XCODE**, em <https://developer.apple.com/documentation/xcode>
- **Writing code with intelligence in Xcode**, em <https://developer.apple.com/documentation/xcode/writing-code-with-intelligence-in-xcode>
- **SWIFT PLAYGROUNDS**, em <https://www.apple.com/swift/playgrounds/>
- **THE SWIFT PROGRAMMING LANGUAGE – SWIFT 5.5**, em <https://docs.swift.org/swift-book/LanguageGuide/TheBasics.html>

Programação Para Dispositivos Móveis II

INTRODUÇÃO À PROGRAMAÇÃO EM SWIFT

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jmr@estg.ipp.pt, Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt